

# Smittotalet

Epidemiskt reproduktionstal ( $R_t$ ) som portföljgasreglage på en TSMOM-basbok

**DOD (Steg 3: redundant med count-EWMA/GARCH-persistens)**

Fullständig kod, dokumentation och resultat  
Genererad av Claude Code

# Innehåll

---

1. README
2. REPORT (resultat och verdikt)
3. Kallkod
4. config.py
5. eodhd\_client.py
6. fetch\_data.py
7. data.py
8. events.py
9. signal.py
10. scheduling.py
11. tsmom.py
12. costs.py
13. backtest.py
14. twins.py
15. battery.py
16. episodes.py
17. nulls.py
18. grid.py
19. metrics.py
20. run\_research.py
21. tests/test\_config.py
22. tests/test\_events.py
23. tests/test\_signal.py
24. tests/test\_tsmom.py
25. tests/test\_costs.py
26. tests/test\_twins.py
27. tests/test\_episodes.py
28. tests/test\_nulls.py
29. tests/test\_metrics.py

# Smittotalet ("the infection count")

Se REPORT.md för resultat och verdikt.

## Hypotesen

Ett epidemiskt effektivt reproduktionstal ( $R_t$ , Cori/EpiEstim-metoden) som portfoljgasreglage ovanpå en TSMOM-basbok. Volatilitetsexceedanser över tillgångar behandlas som en forgreningsprocess: varje handelse "foder" framtida handelser via en fix, FRYST karna  $w$ , lanad från Efterskalvsklockans (research/omori) egen verifierade, poolade Omori-fit -- inte nyskattad har.  $R_t$  = kvoten mellan observerade handelser och vad dagens handelser predicerar via den karnan;  $G_t = \text{clip}((1/R_t)^\kappa, 0.3, 1.3)$ , en TILT (aldrig gate, Dammluckans-lardomen) på den redan volskalade basboken.

Se toppniva-uppdraget (task description i denna sessions historik) för den fullständiga, förregistrerade specifikationen -- regler, nollhypoteser, forkastningskriterier, förväntade svagheter (rankade) -- vilken denna implementation följer i sin helhet.

Smittotalet är sjätte laget av samma allmänna EODHD US-ETF-yta/IS-datumintervall i denna repo-serie (efter Dammluckan, Efterskalvsklockan, Vindkastet, Runraden, Oglegrinden) -- DSR-korrigeringen i grid.py/run\_research.py rakenskar detta explicit (se config.N\_EFFECTIVE\_SURFACE\_READS).

## Vad som INTE fanns fardigt i repot (och därför byggdes har från grunden)

Research-agenter som undersökte de fem syskongrenarna innan implementationen inleddes visade att flera saker uppdragstexten hänvisar till som "repots" eller en syskonstrategis "levande komponent" antingen inte existerar, eller existerar i en annan form än texten antyder:

- **"Repots dokumenterade TSMOM-proxy (Vindkastets levande komponent)":** Vindkastets egen handelshypotes (ridge-VAR/ transient growth) är DOD. Den enda TSMOM-koden i den grenen är en outestad akademisk 12m-tecken/invers-20d-vol-proxy, dagligen ombalanserad, 100% brutto, utan volmalning -- använd där bara som korrelationskontroll. Den proxyn är basen för `tsmom.py` har, men veckoombalans + 10%-volmalning + 200%-bruttotak är byggt från grunden (protokoll- regeln "portfoljniva-volskalning ligger i basen" fanns inte fardigimplementerad någonstans).
- **"Oglegrinden-regeln":** existerar inte som en namngiven, numrerad regel i den grenen -- den är denna implementations egen glans på Oglegrindens pre-registrerade forkastningskriterium (b): en grind på dod alfa är fortfarande dod. Används här bokstavligt som gate 2. Kalles har "Oglegrinden-regeln" för att undvika icke-ASCII-tecken i Python-identifierare.
- **"Dammluckans matchade-bredd-krav" / kvantilmappning:** existerar inte bokstavligt i Dammluckan (ingen kvantilmappning, ingen orakel-tak-koncept där alls). `twins.quantile_map_to` och `backtest.oracle_cap_test` är därför nybyggda här, med Dammluckans metodologiska disciplin (matchad-maskineri-kontroll, se Dammluckans donchian-tvilling) som forebild, inte som en portering.
- **"Runradens mallkrav 1/4":** existerar inte som numrerad lista i Runraden -- K1b är den facto PC1-pa-input-kontrollen (mallkrav 4-analogen), och "minimieffektgolv" är en las från Runradens egen efterhandsanalys (K1a/K2 klarar nollhypotesen men landar själva på ett värde som praktiskt taget är noll) snarare än en existerande funktion. `battery.py` implementerar bada explicit här (REDUNDANCY\_MIN\_DELTA\_R2 utöver R2-tröskeln).
- **Omori-karnans frysta "c":** Efterskalvsklockan profilerar  $c$  per handelse över griden  $\{0,1,2\}$ ; ingen enskild fryst POOLAD  $c$  rapporteras. DECLARED:  $c=1$  (grid-mittpunkt),  $p=0.5758...$  (den faktiska frysta, poolade globalpriorn från `research/omori/output/priors.json`).

Se config.py för varje enskild DECLARED-flaggad konstant med motivering.

## Modulkarta

| Fil                          | Ansvar                                                                                                          |
|------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <code>config.py</code>       | Alla brief-pinnade OCH deklarerade konstanter. Varje icke-brief-pinnad konstant är kommenterad som DECLARED.    |
| <code>eodhd_client.py</code> | EODHD-klient, portad från <code>research/runraden/eodhd_client.py</code> .                                      |
| <code>fetch_data.py</code>   | Hamtar IS_UNIVERSE-panelen, cachear kombinerade per-falt-CSV:er (samma monster som dammluckan/omori).           |
| <code>data.py</code>         | Panel-laddning från CSV.                                                                                        |
| <code>events.py</code>       | $E_{\{i,t\}} = 1\{ r  > \text{rullande } 252\text{d-percentil } q, \text{ PIT } t-1\}$ ; $X_t = \text{summa}$ . |

| Fil             | Ansvar                                                                                                                                                   |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| signal.py       | Frysta Omori-karnan $w_s$ , $\Lambda_t$ , $\text{Cori } R_{\text{hat}_t}$ (sluten form, $\text{Gamma}(1,1)$ -prior), $G_t$ -tilten.                      |
| tsmom.py        | Basboken: 12m-tecken/invers-20d-vol, veckoombalans, iterativ k-lösning mot 10% volmal under 200%-bruttotak (Dammluckans fixade-kalibreringsvag-monster). |
| scheduling.py   | Delad fredags-lag-tillämpningshjälpare (både bas och överlag delar samma veckokadens).                                                                   |
| costs.py        | ADV-bucket-kostnadsmodell, portad från <code>research/dammLuckan/costs.py</code> .                                                                       |
| backtest.py     | Kopplar ihop överlaget på basboken + orakel-tak-testet.                                                                                                  |
| twins.py        | T1 (voltarget), T2 (count-EWMA, "den brutala"), T3 (broadcast a la Runradens T4) -- alla kvantilmappade till $G_t$ 's egen ovillkorliga fördelning.      |
| battery.py      | Redundansscreen: $R_{\text{hat}}$ mot count-EWMA + GARCH-persistensproxy, både direkt-R2 och inkrementell-mot-framatblickande-vol.                       |
| episodes.py     | Superkritiska episoder ( $\geq 5d$ , $\geq 21d$ separation) + bindande $G < 0.7$ -andel.                                                                 |
| nulls.py        | Cirkulart blockshuffle (dispersionsnull på $R_{\text{hat}}$ ) + block-bootstrap-CI (Steg 5).                                                             |
| grid.py         | 8-cells grid ( $q \times \tau \times \kappa$ ), teckenstabilitet, DSR från grid, +50% karnrobusthet utanför griden.                                      |
| metrics.py      | Sharpe, max drawdown, PSR/DSR (Bailey & Lopez de Prado), NW-t, delvis-period-teckenkonsistens.                                                           |
| run_research.py | Orkestrerar hela pipeline i förregistrerad kostnadsordning (orakel -> basmotor -> redundans -> dispersion/episoder -> Steg 5 -> lasst OOS).              |

## Kora

```

pip install -r research/smittotalet/requirements.txt
export EODHD_API_KEY=...
python -m research.smittotalet.run_research
pytest research/smittotalet/tests -q

```

# Smittotalet -- resultat och verdict

**Verdict: FÖRKASTAD.** Dör vid Steg 3 (redundansscreen), det billigaste och mest sannolika grav-scenariot uppdragstexten själv rankade högst ("(1) Redundans ... Mest sannolika graven, och den billigaste"). Bekräftat oberoende av Steg 4 (dispersionsnull), som aldrig nås i den förregistrerade ordningen men som beräknats separat här för transparens och pekar åt samma håll.

## 0. Data

EODHD dagliga OHLCV, IS\_UNIVERSE (40 tickers, pinnad ur Efterskalvsklockans repo, se config.py), 2003-01-01 -> senast hämtade bar. IS = 2004-01-01 -- 2017-12-31 (samma gräns som Dammluckan/Efterskalvsklockan använder på samma yta). Alla 40 tickers hämtades utan fel.

## 1. Orakel-tak (Steg 1) -- KLARAR

Rearrangement-inequality-orakel (samma G-fördelning som primären, omordnad i tid för att maximera  $\sum(G^*r)$  mot basbokens faktiska veckoavkastning -- se `backtest.oracle_cap_test`):

|                              | Sharpe (veckovis, annualiserad, IS 2004-2017) |
|------------------------------|-----------------------------------------------|
| Bas (TSMOM, netto kostnader) | 0.571                                         |
| Orakel-G                     | 2.293                                         |
| <b>Inkrement</b>             | <b>+1.722</b> (kräver $\geq +0.15$ )          |

Reglageklassen har gott om tak här -- perfekt framförhållning på G hade varit mycket värdefull. Detta säger inget om huruvida Smittotalets FAKTISKA  $R_t$ -skattning kommer nära det taket; det är precis vad efterföljande steg testar.

## 2. Basmotor (Steg 2, "Öglegrinden-regeln") -- KLARAR

TSMOM-sleeven (12m-tecken, invers-20d-vol, veckoombalans, 10%-volmål under 200%- bruttotak, k löst iterativt IS = 0.000461) ger **IS-Sharpe 0.533 netto kostnader** > 0. Basmotorn är inte död alfa -- en grind (eller tilt) här har något att stå på.

## 3. Redundansscreen (Steg 3) -- DÖR HÄR

$R_{\hat{t}}$  regresserat mot [count-EWMA( $\tau$ ), GARCH(0.94)-EWMA-varians-proxy på sleeveens poolade |avkastning|] (HAC-robusta OLS, se `battery.py`):

Alla siffror strikt IS (2004-01-01 -- 2017-12-31); OOS förblir låst till Steg 5 nås, vilket den inte gör här.

| Kontroll                                    | Värde         | Tröskel                             | Utfall                         |
|---------------------------------------------|---------------|-------------------------------------|--------------------------------|
| Direkt $R^2$ ( $R_{\hat{t}}$ ~ kontroller)  | 0.075         | > 0.5 dödar                         | Klarar (inte redundant i nivå) |
| Kontroller- $R^2$ (framåtblickande 21d-vol) | 0.500         | --                                  | --                             |
| Full- $R^2$ (kontroller + $R_{\hat{t}}$ )   | 0.503         | --                                  | --                             |
| <b>Inkrementell <math>\Delta R^2</math></b> | <b>0.0033</b> | <b><math>\geq 0.01</math> krävs</b> | <b>Dödar</b>                   |
| NW-t på $R_{\hat{t}}$ -koefficienten        | 2.07          | $ t  \geq 1.96$                     | Statistiskt signifikant...     |

$R_{\hat{t}}$  är INTE en enkel linjär omskrivning av count-EWMA/vol-klustring i nivå (direkt- $R^2$  klarar gott och väl). Men dess EGET bidrag till att predicera framåtblickande realiserad volatilitet, utöver vad count-EWMA och GARCH-persistens redan ger (kontrollerna ensamma förklarar redan hälften av variationen i framåtblickande vol,  $R^2=0.50$ ), är ekonomiskt försumbart ( $\Delta R^2 = 0.33$  procentenheter) trots att koefficienten är statistiskt skild från noll (NW-t = 2.07, precis över 1.96). Detta är exakt Runradens egen efterhandslärdom (K1a/K2 "klaras en tröskel som i praktiken ligger vid noll") -- ett rent p-värde-krav hade felaktigt släppt igenom denna signal; minimieffektgolvet (REDUNDANCY\_MIN\_DELTA\_R2) fångar det.

## 4. Dispersionsnull + episodräkning (Steg 4) -- ALDRIG FORMELLT NÅDD, beräknad separat

---

Beräknad för transparens (inte del av det förregistrerade beslutet, eftersom pipelinen redan stoppat vid Steg 3):

- $R_{\hat{t}}$  IS: median 0.994, IQR [0.826, 1.175], std 0.360.
- Real dispersion (std av  $R_{\hat{t}}$ , IS) = **0.360**; block-shuffle-nullens 95:e percentil (300 drag, blocklängd 21d) = **0.510**. **Real dispersion < null-p95 -- hade dödat pipelinen även om Steg 3 klarats.**  $R_{\hat{t}}$ :s tidsvariation är inte urskiljbar från vad slumpmässigt omblandade händelseblock producerar.
- Episodräkning: 58 superkritiska episoder ( $R_{\hat{t}} > 1$ ,  $\geq 5d$ ,  $\geq 21d$  separation) -- klarar golvet på  $\geq 10$  med bred marginal.
- Bindande andel (veckor med  $G < 0.7$ ): 9.6% -- inom [5%, 30%]-intervallet.

Episodantalet och den bindande andelen är alltså inte problemet -- reglaget SLÅR TILL tillräckligt ofta. Problemet är att dess timing/amplitud inte skiljer sig från brus när den ställs mot en blockad nollhypotes, vilket är samma slutsats redundansscreenen redan nått via en annan väg.

## 5+ (aldrig körd)

---

Steg 5 (tvilling-inkrement, bootstrap-CI, teckenkonsistens, DSR), det fulla 8-cells- griden, +-50%-kärnrobusthetskontrollen och den låsta OOS-avläsningen (2018-) är implementerade och testade (`grid.py`, `twins.py`, `nulls.block_bootstrap_sharpe_ci`, `run_research._run_oos`) men aldrig körda på riktig data, eftersom Steg 3 redan avgjorde utfallet -- exakt samma disciplin som Runradens `steg2.py` ("implemented and unit-tested but never run on real data since Steg 0 failed").

## Rotorsak, rankad (jämfört med uppdragstextens egen förhandsrankning)

---

Uppdragstexten rankade "(1) Redundans ... billigaste graven" som mest sannolik, före "(2) Latens" och "(3) Episodfattigdom". Utfallet bekräftar rankningen exakt:

1. **Bekräftad.**  $R_{\hat{t}}$  är, framåtblickande, i praktiken en (svagt förstärkt) omskrivning av count-EWMA/vol-klustring. Cori-kvotstrukturen (dividera med den frysta Omori-kärnans implicerade förväntan) tillför signifikant men ekonomiskt försumbar inkrementell information utöver den råa nivån.
2. **Ej testad formellt** (pipelinen stoppade före Steg 4), men den beräknade dispersionsnull-kontrollen pekar åt samma håll:  $R_{\hat{t}}$ :s egen tidsvariation är inte skild från ett blockshufflat brusband. Om Steg 3 hade klarats, hade Steg 4 dödat ändå.
3. **Motbevisad.** Episodfattigdom var INTE problemet -- 58 episoder mot golvet på 10, gott om marginal. Bredd fanns; det den saknade var särskiljbarhet.

## Slutsats

---

Smittotalet dör där uppdragstexten förutspådde att den mest sannolikt skulle dö, och av samma skäl uppdragstexten själv namngav som mest sannolikt: en Cori/EpiEstim-reproduktionstalsskattning på poolade tvärsnittshändelser är, i denna implementation, inte urskiljbar från dess billigaste tvilling (count-EWMA + GARCH-persistens) när den ställs mot en framåtblickande volatilitetsprediktion. Basmotorn och orakeltaket visar att reglageklassen i sig hade utrymme att tillföra värde på denna sleeve -- det är den specifika  $R_{\hat{t}}$ -konstruktionen, inte TSMOM-boken eller regleragekonceptet i stort, som misslyckas här.

## config.py

```
"""
Smittotalet ("the infection count") -- shared configuration.

An epidemic effective-reproduction-number ( $R_t$ ) overlay on a TSMOM base book.
Positioned against Efterskalvsklockan (omori): same self-exciting-process
phenomenon family, opposite estimand. Efterskalvsklockan asked "does this
event have its own clock?" (per-instrument Omori decay, answer: no -- the
strategy was REJECTED). Smittotalet asks "how loaded is the whole system
right now?" and uses Efterskalvsklockan's frozen, verified pooled-event
Omori-decay kernel as a FIXED INPUT (the serial-interval kernel  $w_s$ ), not as
something to re-estimate.

House convention (Dammluckan / Efterskalvsklockan / Runraden / Vindkastet /
Oglegrinden): every constant that is not literally pinned by the brief is
flagged DECLARED, with a one-line rationale. Nothing here is silently
assumed.
"""
import os
from dataclasses import dataclass, field

# -----
# Universe -- "IS = 40-ETF-panelen (tickerlista pinnad ur Efterskalvsklockans
# repo -- och verifieras mot Runradens rekonstruktion, per registerluckan
# baada gravarna flaggar)"
# -----
# Pinned verbatim from research/omori/config.py:IS_UNIVERSE (Efterskalvsklockan).
IS_UNIVERSE = [
    # Broad US equity (6)
    "SPY", "QQQ", "IWM", "IJH", "VTI", "VUG",
    # Broad international, diversified only (4)
    "EFA", "EEM", "VEA", "VWO",
    # Sector SPDRs (11)
    "XLF", "XLK", "XLE", "XLI", "XLY", "XLP", "XLV", "XLU", "XLB", "XLRE", "XLC",
    # Industry / thematic (5)
    "SMH", "XBI", "KRE", "XRT", "IYT",
    # Fixed income (9)
    "TLT", "IEF", "SHY", "LQD", "HYG", "TIP", "MUB", "AGG", "EMB",
    # FX (3)
    "UUP", "FXE", "FXV",
    # REIT (1)
    "VNQ",
    # Diversified commodity (1)
    "DBC",
] # 40 tickers

# CAVEAT (registerluckan, flagged by BOTH prior "graves" that touched this
# panel): Runraden's own README states "No survivorship-bias handling beyond
# what EODHD serves for the listed tickers; all 40 IS tickers are currently-
# listed, still-trading ETFs" and its precursor Formdriften's REPORT.md
# independently flags "a small residual survivorship bias versus a true PIT
# vendor panel". Smittotalet inherits the same gap: this panel is fetched as
# of today, not reconstructed point-in-time, so any ETF that delisted or was
# renamed inside the sample window is silently absent. Verified against
# Runraden's own reconstruction (research/runraden/config.py:IS_UNIVERSE,
# also ~40 EODHD-native US ETFs): 29/40 tickers are shared between the two
# independently-built panels (see tests/test_config.py), which is the
# cross-check the brief asks for -- it is not, and cannot be, a fix for the
# underlying registry gap.

# -----
# Sample split. DECLARED: mirrors the IS/OOS-1 boundary used by every sibling
# branch that touches this same EODHD panel (Dammluckan, Efterskalvsklockan),
# so Smittotalet's IS design is honestly "already contaminated" like theirs,
# and its OOS read stays comparable. The brief itself proposes exactly this
# split under "OOS-val" ("... den slitna 2018-26-multi-asset-ytan").
HISTORY_START = "2003-01-01" # warm-up buffer before IS_START (252d TSMOM
# lookback + 252d event-threshold window)
IS_START = "2004-01-01"
IS_END = "2017-12-31"
OOS_START = "2018-01-01"
OOS_END = None # None = through the latest fetched bar

TRADING_DAYS_YEAR = 252

# -----
# Event definition -- " $E_{i,t} = 1\{r_{i,t}\} > \text{rullande 252d-percentil } q,$ 
# PIT  $t-1$ ]"
# -----
EVENT_LOOKBACK = 252 # rolling window for the percentile threshold
# PIT discipline (house convention, same as omori/data.py): the threshold at
#  $t$  is computed on  $\text{returns.shift(1).rolling(EVENT\_LOOKBACK)}$ , i.e. strictly
#  $t-252..t-1$ , never touching  $r_t$  itself.

# -----
# Frozen Omori kernel -- "w kalibreras en gang IS med Efterskalvsklockans
```

```

# verifierade Omori-fitter paa poolade haendelser, sedan fryses"
# -----
# The exponent is Efterskalvsklockan's own frozen POOLED (population-level,
# cross-instrument) Omori prior -- research/omori/output/priors.json:"global"
# -- i.e. the one calibrated-once, frozen-before-OOS number in that repo that
# is literally a fit "on pooled events" rather than a per-instrument fit.
# Per-instrument priors exist in that same file but are explicitly NOT what
# the brief asks for here.
OMORI_KERNEL_P = 0.5758457103777312 # frozen, verbatim from omori/output/priors.json
# DECLARED: omori's own Omori-law offset "c" is profiled per-event over the
# grid {0, 1, 2} (omori/priors.py) -- no single frozen *pooled* c is reported
# anywhere in that repo (c is a per-fit nuisance parameter, not a population
# prior). We take the midpoint of that same profiling grid, c=1, as the one
# frozen value, and normalize w_s to sum to 1 over the 10-day kernel so that
# Lambda_t is a weighted average of recent X (EpiEstim/Cori serial-interval
# convention: the serial-interval distribution integrates to 1).
OMORI_KERNEL_C = 1.0
OMORI_KERNEL_LAGS = 10 # s = 1..10 in Lambda_t = sum_s w_s X_{t-s}
OMORI_KERNEL_ROBUSTNESS_PCT = 0.5 # +-50% perturbation of OMORI_KERNEL_P,
# sensitivity check outside the main grid

# -----
# Cori R_t estimator and G_t tilt
# -----
# R_hat_t = (1 + sum_{u=t-tau+1..t} X_u) / (1 + sum_u Lambda_u), gamma(1,1) prior
TAU_GRID = (10, 21)
KAPPA_GRID = (1, 2) # tilt exponent in G_t = clip((1/R_hat)^kappa, .3, 1.3)
Q_GRID = (90, 95) # event-threshold percentile
G_CLIP_LOW = 0.3
G_CLIP_HIGH = 1.3
REBALANCE_WEEKDAY = "FRI" # G_t updated Friday, applied to next ISO week

# -----
# Base engine -- "repots dokumenterade TSMOM-proxy (Vindkastets levande
# komponent)": Vindkastet's own signal hypothesis (ridge-VAR/transient-growth)
# is DEAD (research/vindkastet/REPORT.md: "DOED"); the only TSMOM code in that
# branch is the academic 12m-sign / inverse-20d-vol proxy documented in its
# REPORT.md ("Substituerat med en enkel akademisk 12-manaders TSMOM-portfolj
# ... standardproxy i litteraturen (Moskowitz-Ooi-Pedersen-stil)"), used
# there only as an un-gated correlation benchmark. That signal, with
# portfolio-level vol targeting and a gross cap added on top (the brief's own
# parenthetical: "protokollregeln om portfoljniva-volskalning ligger alltsaa
# i basen, fore overlagget"), is what "the repo's documented TSMOM proxy" is
# taken to mean here.
TSMOM_LOOKBACK = 252 # 12-month sign momentum
TSMOM_VOL_LOOKBACK = 20 # inverse-vol sizing window, verbatim from
# vindkastet/run_backtest_summary.py:trend_proxy_returns
PORTFOLIO_VOL_TARGET = 0.10 # 10% annualized ex-ante, per the brief
GROSS_CAP = 2.0 # 200%, per the brief
VOL_TARGET_SOLVE_MAX_ITER = 25 # Dammluckan's iterative k-solve pattern
VOL_TARGET_SOLVE_TOL = 1e-4

# -----
# Base-engine gate -- "Steg 1 basmotor: TSMOM-sleeven sjaelv maaste ha
# netto-alfa paa IS-panelen (Oeglegrinden-regeln)"
# -----
# Oglegrinden's own literal wording (REPORT.md): "en grind paa doed alfa aer
# fortfarande doed" -- a gate on dead alpha is still dead. No numeric bar is
# named there beyond "positive net-of-cost Sharpe IS"; DECLARED: we use that
# qualitative bar literally, ann_sharpe(base, IS, net of costs) > 0.
BASE_ENGINE_MIN_IS_SHARPE = 0.0

# -----
# Costs -- "repots block-bootstrap och ADV-bucket-kostnader (ej Runradens
# platta 2 bp)". Ported verbatim from research/dammluckan/costs.py, which is
# itself ported verbatim from research/formdriften/costs.py -- the only
# ADV-bucketed cost model anywhere in this repo's research-branch series.
# -----
COMMISSION_BPS = 3.0
ADV_BUCKETS = ( # (min ADV $, half-spread bps), most liquid first
    (500e6, 0.5),
    (200e6, 1.0),
    (100e6, 1.5),
    (50e6, 2.5),
    (20e6, 4.0),
    (0.0, 7.0),
)
ADV_COST_LOOKBACK = 63
# DOCUMENTED SPREAD CAVEAT (verbatim convention, restated here as in every
# sibling that reuses this cost model): EODHD does not provide quoted
# bid/ask spread history, so the half-spread leg is approximated from
# trailing dollar-ADV via a monotone liquidity bucket -- a documented
# approximation, not a measured spread. Only the commission leg (3bp/side)
# is a clean, un-approximated assumption.

# -----
# Redundancy screen kill threshold ("R^2 > 0.5 eller inkrementell delta-R^2 /
# NW-t ~ 0 for framatblickande vol/drawdown -> doed"). DECLARED minimum-effect
# floor paired with the null-percentile comparison (Runraden's own post-mortem
# lesson: "K1a/K2 pass but clear a threshold that itself sits essentially at
# zero" -- a percentile-only gate is not enough).

```



```

REDUNDANCY_R2_KILL = 0.5
REDUNDANCY_MIN_DELTA_R2 = 0.01      # incremental R^2 must clear this AND be
                                     # NW-significant to count as "not dead"

# -----
# Dispersion / episode-count gate
# -----
EPISODE_MIN_RUN_DAYS = 5             # R_hat > 1 for >= 5 consecutive days
EPISODE_MIN_SEPARATION_DAYS = 21     # >= 21 days between episodes to count distinct
EPISODE_MIN_COUNT_IS = 10
BINDING_G_LOW_THRESHOLD = 0.7        # share of weeks with G < 0.7
BINDING_G_LOW_SHARE_RANGE = (0.05, 0.30)

# -----
# Step 2 acceptance bar
# -----
STEP2_MIN_SR_INCREMENT = 0.10
STEP2_MIN_SIGN_CONSISTENCY = 0.60
N_SUBPERIODS = 2                     # "teckenkonsistens mellan IS-halvor" -- 2 halves
DSR_OOS_KILL = 0.0
ORACLE_CAP_MIN_SR_INCREMENT = 0.15   # "ger den < +0.15 netto-SR over basen ar
                                     # hela reglageklassen doed" -- cheapest,
                                     # first gate

# "DSR >= 0 inklusive poolad korrektion for ytans tidigare avlasningar":
# DECLARED count of prior strategies in this repo's research-branch series
# that already spent selection budget on this same general EODHD US-ETF
# panel / IS date range: Dammluckan, Efterskalvsklockan (omori), Vindkastet,
# Runraden, Oglegrinden = 5 prior reads. Total effective reads including
# Smittotalet itself = 6. Used to inflate the DSR trial-pool SIZE (not its
# spread) when computing the pooled-surface-corrected DSR, by tiling the
# grid's own Sharpe trials -- tiling preserves std(sr_trials) exactly while
# raising n, which is what expected_max_sharpe's order-statistics
# approximation actually consumes.
N_PRIOR_SURFACE_READS = 5
N_EFFECTIVE_SURFACE_READS = N_PRIOR_SURFACE_READS + 1

# -----
# EODHD access -- env var convention pinned by the eodhd-api-key-setup branch,
# HTTP client ported from research/runraden/eodhd_client.py.
# -----
EODHD_API_KEY_ENV = "EODHD_API_KEY"
_THIS_DIR = os.path.dirname(os.path.abspath(__file__))
CACHE_DIR = os.path.join(_THIS_DIR, "data", "raw")
DATA_DIR = os.path.join(_THIS_DIR, "data")
OUTPUT_DIR = os.path.join(_THIS_DIR, "output")

FIELDS = ("open", "high", "low", "close", "adjusted_close", "volume")

@dataclass(frozen=True)
class GridCell:
    q: int
    tau: int
    kappa: int

GRID = tuple(
    GridCell(q=q, tau=tau, kappa=kappa)
    for q in Q_GRID
    for tau in TAU_GRID
    for kappa in KAPPA_GRID
) # 2 x 2 x 2 = 8 cells

DEFAULT_CELL = GridCell(q=95, tau=21, kappa=1)

```

## eodhd\_client.py

```
"""Thin EODHD EOD data client with on-disk parquet caching.

Ported from research/runraden/eodhd_client.py (same API surface, same
caching convention). Only fetches adjusted daily OHLCV. No point-in-time
survivorship handling is attempted beyond what EODHD serves for the ticker
list supplied by the caller -- see config.py's "registerluckan" caveat.
"""

from __future__ import annotations

import os
import time
from typing import Iterable

import pandas as pd
import requests

from . import config

EODHD_BASE = "https://eodhd.com/api/eod"

class EODHDError(RuntimeError):
    pass

def _api_key() -> str:
    key = os.environ.get(config.EODHD_API_KEY_ENV)
    if not key:
        raise EODHDError(f"{config.EODHD_API_KEY_ENV} not set in environment")
    return key

def _cache_path(ticker: str) -> str:
    os.makedirs(config.CACHE_DIR, exist_ok=True)
    safe = ticker.replace("/", " ")
    return os.path.join(config.CACHE_DIR, f"{safe}.parquet")

def fetch_eod(ticker: str, start: str = "1990-01-01", end: str | None = None,
              force_refresh: bool = False, max_retries: int = 4) -> pd.DataFrame:
    """Fetch daily adjusted OHLCV for `ticker`, cached to disk as parquet.

    Returns a DataFrame indexed by date with columns
    [open, high, low, close, adjusted_close, volume].
    """
    cache_file = _cache_path(ticker)
    if not force_refresh and os.path.exists(cache_file):
        return pd.read_parquet(cache_file)

    key = _api_key()
    params = {
        "api_token": key,
        "fmt": "json",
        "period": "d",
        "from": start,
    }
    if end:
        params["to"] = end

    url = f"{EODHD_BASE}/{ticker}.US"
    last_err: Exception | None = None
    for attempt in range(max_retries):
        try:
            resp = requests.get(url, params=params, timeout=60)
            if resp.status_code == 200:
                payload = resp.json()
                if not isinstance(payload, list) or len(payload) == 0:
                    raise EODHDError(f"Empty/invalid payload for {ticker}: {str(payload)[:200]}")
                df = pd.DataFrame(payload)
                df["date"] = pd.to_datetime(df["date"])
                df = df.set_index("date").sort_index()
                cols = list(config.FIELDS)
                df = df[[c for c in cols if c in df.columns]]
                df.to_parquet(cache_file)
                return df
            last_err = EODHDError(f"HTTP {resp.status_code} for {ticker}: {resp.text[:200]}")
        except Exception as exc: # noqa: BLE001
            last_err = exc
        if attempt < max_retries - 1:
            time.sleep(2 ** attempt)
    raise EODHDError(f"Failed to fetch {ticker} after {max_retries} attempts: {last_err}")

def fetch_panel(tickers: Iterable[str], start: str = "1990-01-01", end: str | None = None,
                force_refresh: bool = False) -> dict[str, pd.DataFrame]:
```

```

panel: dict[str, pd.DataFrame] = {}
failures: dict[str, str] = {}
for ticker in tickers:
    try:
        panel[ticker] = fetch_eod(ticker, start=start, end=end, force_refresh=force_refresh)
    except EODHDError as exc:
        failures[ticker] = str(exc)
if failures:
    print(f"[eodhd_client] WARNING: {len(failures)} tickers failed to fetch:")
    for t, msg in failures.items():
        print(f"    {t}: {msg}")
return panel

```

## fetch\_data.py

```
"""Fetch the IS_UNIVERSE panel from EODHD and cache combined per-field CSVs.

Mirrors the sibling convention (omori/dammluckan): one wide CSV per OHLCV
field under data/, columns = tickers, index = date. Run once; downstream
code reads the CSVs, not EODHD directly.
"""
import os

import pandas as pd

from . import config
from . import eodhd_client

def build_panel_csvs(tickers=None, start=None, end=None, force_refresh=False):
    tickers = tickers or config.IS_UNIVERSE
    start = start or config.HISTORY_START
    raw = eodhd_client.fetch_panel(tickers, start=start, end=end, force_refresh=force_refresh)

    os.makedirs(config.DATA_DIR, exist_ok=True)
    for field in config.FIELDS:
        cols = {}
        for ticker, df in raw.items():
            if field in df.columns:
                cols[ticker] = df[field]
        if not cols:
            continue
        wide = pd.DataFrame(cols).sort_index()
        wide.to_csv(os.path.join(config.DATA_DIR, f"primary_{field}.csv"))

    fetched = sorted(raw.keys())
    missing = sorted(set(tickers) - set(fetched))
    print(f"[fetch_data] fetched {len(fetched)}/{len(tickers)} tickers"
          + (f", missing: {missing}" if missing else ""))
    return raw

if __name__ == "__main__":
    build_panel_csvs()
```

## data.py

```
"""Panel loader -- reads the cached per-field CSVs built by fetch_data.py."""
import os
from dataclasses import dataclass

import numpy as np
import pandas as pd

from . import config

@dataclass
class Panel:
    open: pd.DataFrame
    high: pd.DataFrame
    low: pd.DataFrame
    close: pd.DataFrame
    adjusted_close: pd.DataFrame
    volume: pd.DataFrame

    @property
    def tickers(self):
        return list(self.adjusted_close.columns)

    def simple_returns(self) -> pd.DataFrame:
        return self.adjusted_close.pct_change()

    def dollar_volume(self) -> pd.DataFrame:
        return self.close * self.volume

    def adv(self, lookback=config.ADV_COST_LOOKBACK) -> pd.DataFrame:
        return self.dollar_volume().shift(1).rolling(lookback).mean()

def _load_field(field: str) -> pd.DataFrame:
    path = os.path.join(config.DATA_DIR, f"primary_{field}.csv")
    return pd.read_csv(path, index_col=0, parse_dates=True)

def load_primary_panel() -> Panel:
    fields = {f: _load_field(f) for f in config.FIELDS}
    common_cols = set.intersection(*(set(df.columns) for df in fields.values()))
    common_idx = fields["close"].index
    for df in fields.values():
        common_idx = common_idx.union(df.index)
    aligned = {
        f: df.reindex(index=sorted(common_idx), columns=sorted(common_cols))
        for f, df in fields.items()
    }
    return Panel(
        open=aligned["open"], high=aligned["high"], low=aligned["low"],
        close=aligned["close"], adjusted_close=aligned["adjusted_close"],
        volume=aligned["volume"],
    )

def restrict(panel: Panel, start=None, end=None) -> Panel:
    def _cut(df):
        return df.loc[(slice(start, end)), :]
    return Panel(
        open=_cut(panel.open), high=_cut(panel.high), low=_cut(panel.low),
        close=_cut(panel.close), adjusted_close=_cut(panel.adjusted_close),
        volume=_cut(panel.volume),
    )
```

## events.py

```
"""Volatility-exceedance event detection.

 $E_{i,t} = 1\{|r_{i,t}| > \text{rolling } 252d \text{ percentile } q, \text{ PIT } t-1\}$ ;  $X_t = \sum_i E_{i,t}$ .

PIT discipline (house convention, same as omori/data.py): the threshold at  $t$ 
is computed on  $\text{abs}(\text{returns}).\text{shift}(1).\text{rolling}(\text{lookback})$ , i.e. strictly the
window  $t\text{-lookback}..t-1$ , never touching  $r_t$  itself.
"""
import numpy as np
import pandas as pd

from . import config

def event_threshold(returns: pd.DataFrame, q: int, lookback: int = config.EVENT_LOOKBACK) -> pd.DataFrame:
    """Rolling  $q$ -th percentile of  $|returns|$ , evaluated strictly on  $t\text{-lookback}..t-1$ ."""
    abs_r = returns.abs()
    return abs_r.shift(1).rolling(lookback, min_periods=lookback).quantile(q / 100.0)

def event_matrix(returns: pd.DataFrame, q: int, lookback: int = config.EVENT_LOOKBACK) -> pd.DataFrame:
    """Boolean-as-int  $E_{i,t}$  matrix. NaN wherever the PIT threshold isn't yet defined."""
    abs_r = returns.abs()
    thresh = event_threshold(returns, q, lookback)
    events = (abs_r > thresh).astype(float)
    events[thresh.isna()] = np.nan
    return events

def daily_count(events: pd.DataFrame) -> pd.Series:
    """ $X_t = \sum_i E_{i,t}$ , counting only assets with a defined threshold that day."""
    return events.sum(axis=1, skipna=True)

def build(returns: pd.DataFrame, q: int, lookback: int = config.EVENT_LOOKBACK):
    events = event_matrix(returns, q, lookback)
    x_t = daily_count(events)
    return events, x_t
```

## signal.py

```
"""Cori (EpiEstim) effective-reproduction-number estimator and the G_t tilt.

    Lambda_t = sum_{s=1..10} w_s X_{t-s} (frozen Omori kernel)
    R_hat_t = (1 + sum_{u=t-tau+1..t} X_u) / (1 + sum_u Lambda_u) (Gamma(1,1) prior)
    G_t = clip((1/R_hat_t)^kappa, 0.3, 1.3)

All closed-form; no MCMC, no fitting at signal-construction time -- the only
fitted object (the Omori kernel exponent) is frozen in config.py, imported
from Efterskalvsklockan's own frozen output.
"""
import numpy as np
import pandas as pd

from . import config

def omori_kernel(lags: int = config.OMORI_KERNEL_LAGS, p: float = config.OMORI_KERNEL_P,
                 c: float = config.OMORI_KERNEL_C) -> np.ndarray:
    """w_s ~ (s+c)^-p for s=1..lags, normalized to sum to 1."""
    s = np.arange(1, lags + 1, dtype=float)
    raw = (s + c) ** (-p)
    return raw / raw.sum()

def branching_intensity(x_t: pd.Series, w: np.ndarray) -> pd.Series:
    """Lambda_t = sum_s w_s X_{t-s}, causal (only uses strictly past X)."""
    lags = len(w)
    lam = pd.Series(0.0, index=x_t.index)
    x_filled = x_t.fillna(0.0)
    for s, w_s in enumerate(w, start=1):
        lam = lam.add(w_s * x_filled.shift(s), fill_value=0.0)
    # Lambda undefined until `lags` full days of prior X exist.
    lam.iloc[:lags] = np.nan
    return lam

def reproduction_number(x_t: pd.Series, lam_t: pd.Series, tau: int) -> pd.Series:
    """R_hat_t, closed-form Cori posterior mean under a Gamma(1,1) prior."""
    x_sum = x_t.fillna(0.0).rolling(tau, min_periods=tau).sum()
    lam_sum = lam_t.fillna(0.0).rolling(tau, min_periods=tau).sum()
    r_hat = (1.0 + x_sum) / (1.0 + lam_sum)
    r_hat[lam_t.isna() | x_t.isna()] = np.nan
    return r_hat

def tilt(r_hat: pd.Series, kappa: float, low: float = config.G_CLIP_LOW,
         high: float = config.G_CLIP_HIGH) -> pd.Series:
    """G_t = clip((1/R_hat_t)^kappa, low, high)."""
    g = (1.0 / r_hat) ** kappa
    return g.clip(lower=low, upper=high)

def build(x_t: pd.Series, q: int, tau: int, kappa: float, p: float = config.OMORI_KERNEL_P,
          c: float = config.OMORI_KERNEL_C, lags: int = config.OMORI_KERNEL_LAGS):
    """Full pipeline: kernel -> Lambda_t -> R_hat_t -> G_t. Returns (r_hat, g_t, lam_t)."""
    w = omori_kernel(lags=lags, p=p, c=c)
    lam_t = branching_intensity(x_t, w)
    r_hat = reproduction_number(x_t, lam_t, tau)
    g_t = tilt(r_hat, kappa)
    return r_hat, g_t, lam_t
```

## scheduling.py

---

```
"""Shared weekly rebalance-timing helper: sample a signal at each week-end
(Friday) close, hold it constant through the FOLLOWING ISO week. Used by
both the TSMOM base book (its own signal) and the G_t overlay (a different
signal, same cadence) so both are lagged by exactly one week, no look-ahead.
"""
import pandas as pd

from . import config

def friday_lag_apply(obj):
    """obj: pd.Series or pd.DataFrame, daily-indexed. Returns the same shape,
    where every day within ISO week W carries the value observed at week
    (W-1)'s Friday close."""
    week_period = obj.index.to_period(f"W-{config.REBALANCE_WEEKDAY}")
    week_end_date = pd.Series(obj.index, index=week_period).groupby(level=0).last()
    at_week_end = obj.loc[week_end_date.values].copy()
    at_week_end.index = week_end_date.index
    applied = at_week_end.shift(1)
    daily = applied.loc[week_period]
    daily.index = obj.index
    return daily

def week_end_values(obj):
    """The value sampled at each week's Friday close (no lag), indexed by
    weekly PeriodIndex -- e.g. for reading off the primary G_t series used
    to quantile-map the null twins onto."""
    week_period = obj.index.to_period(f"W-{config.REBALANCE_WEEKDAY}")
    week_end_date = pd.Series(obj.index, index=week_period).groupby(level=0).last()
    at_week_end = obj.loc[week_end_date.values].copy()
    at_week_end.index = week_end_date.index
    return at_week_end
```



```

"""Base engine: 12-month sign TSMOM, weekly rebalance, portfolio-level vol
targeting to 10% annualized, 200% gross cap.

"Repts dokumenterade TSMOM-proxy (Vindkastets levande komponent)": the
signal itself (sign(P_t/P_{t-252}-1), inverse-20d-vol sizing) is Vindkastet's
own documented academic proxy (research/vindkastet REPORT.md), used there
un-gated, daily-rebalanced, 100%-gross. Here it is promoted to a real base
book: weekly (Friday) rebalance, held one ISO week, portfolio-level vol
targeting, and a 200% gross cap -- "protokollregeln om portfoljniva-
volskalning ligger alltsaa i basen, fore overlagget".

The scaling constant k (raw signal -> target-vol weights) is solved
iteratively, not in one linear step, following Dammluckan's own documented
fix (research/dammluckan/backtest.py: solve_k_for_target_vol): a one-shot
linear rescale is only exact when the gross cap never binds. Because the
200% cap binds on some weeks and not others, realized portfolio vol is a
concave, saturating function of k, not linear in it -- so k is calibrated by
fixed-point iteration against REALIZED IS vol, then frozen for OOS (the
"Dammluckan-fixade kalibreringsvagen" applied here).
"""
import numpy as np
import pandas as pd

from . import config
from . import costs
from . import scheduling

def raw_signal(panel) -> pd.DataFrame:
    """sign(P_t/P_{t-252}-1) / vol20_t, daily, no lag applied yet."""
    adj = panel.adjusted_close
    mom = np.sign(adj / adj.shift(config.TSMOM_LOOKBACK) - 1.0)
    daily_ret = adj.pct_change()
    vol20 = daily_ret.rolling(config.TSMOM_VOL_LOOKBACK).std()
    return mom / vol20

def weekly_rebalanced_position(panel) -> pd.DataFrame:
    """Sample raw_signal at each Friday (week-end) close, hold constant for
    the FOLLOWING ISO week (1-week lag -- no look-ahead)."""
    return scheduling.friday_lag_apply(raw_signal(panel))

def apply_gross_cap(daily_position: pd.DataFrame, k: float, gross_cap: float = config.GROSS_CAP) -> pd.DataFrame:
    scaled = daily_position * k
    gross = scaled.abs().sum(axis=1)
    scale_down = (gross_cap / gross).clip(upper=1.0).fillna(1.0)
    return scaled.mul(scale_down, axis=0)

def _turnover_cost_returns(panel, weights: pd.DataFrame) -> pd.Series:
    adv = panel.adv()
    delta = weights.diff().abs()
    cost_frac = costs.cost_fraction(adv)
    per_name_cost = delta * cost_frac
    return per_name_cost.sum(axis=1, skipna=True).reindex(weights.index).fillna(0.0)

def portfolio_returns(panel, weights: pd.DataFrame, apply_costs: bool = True) -> pd.Series:
    simple_ret = panel.simple_returns()
    gross_ret = (weights * simple_ret).sum(axis=1, skipna=True)
    if apply_costs:
        gross_ret = gross_ret - _turnover_cost_returns(panel, weights)
    return gross_ret

def solve_k_for_target_vol(panel, is_start, is_end, target_vol: float = config.PORTFOLIO_VOL_TARGET,
                           gross_cap: float = config.GROSS_CAP, k0: float | None = None,
                           max_iter: int = config.VOL_TARGET_SOLVE_MAX_ITER,
                           tol: float = config.VOL_TARGET_SOLVE_TOL) -> float:
    raw = weekly_rebalanced_position(panel)
    if k0 is None:
        unscaled = raw.mul(1.0)
        r0 = (unscaled * panel.simple_returns()).sum(axis=1, skipna=True).loc[is_start:is_end]
        vol0 = r0.std() * np.sqrt(config.TRADING_DAYS_YEAR)
        k0 = target_vol / vol0 if vol0 and vol0 > 0 else 1.0
    k = k0
    for _ in range(max_iter):
        weights = apply_gross_cap(raw, k, gross_cap)
        rets = portfolio_returns(panel, weights, apply_costs=False).loc[is_start:is_end]
        vol = rets.std() * np.sqrt(config.TRADING_DAYS_YEAR)
        if not vol or vol <= 0:
            break
        if abs(vol - target_vol) < tol:
            break

```

```

        k = k * (target_vol / vol)
    return k

def build_base_book(panel, k: float, gross_cap: float = config.GROSS_CAP, apply_costs: bool = True):
    """Returns (weights, daily_returns) for the frozen-k base TSMOM book."""
    raw = weekly_rebalanced_position(panel)
    weights = apply_gross_cap(raw, k, gross_cap)
    rets = portfolio_returns(panel, weights, apply_costs=apply_costs)
    return weights, rets

```

## costs.py

```
"""ADV-bucket transaction cost model.

Ported verbatim (bucket thresholds, bps schedule, and the documented spread
caveat) from research/dammluckan/costs.py, itself ported from
research/formdriften/costs.py -- the only ADV-bucketed cost model anywhere
in this repo's research-branch series. This is what the brief means by
"repots ... ADV-bucket-kostnader (ej Runradens platta 2 bp)".
"""
import numpy as np
import pandas as pd

from . import config

_ADV_BUCKETS = config.ADV_BUCKETS # (min ADV $, half-spread bps), most liquid first

def half_spread_bps(adv_dollars):
    """Vectorized monotone ADV-bucket lookup. Accepts scalar, ndarray, or Series."""
    arr = np.asarray(adv_dollars, dtype=float)
    out = np.full(arr.shape, _ADV_BUCKETS[-1][1], dtype=float)
    for min_adv, bps in sorted(_ADV_BUCKETS, key=lambda x: x[0]):
        out = np.where(arr >= min_adv, bps, out)
    if np.isscalar(adv_dollars):
        return float(out)
    if isinstance(adv_dollars, pd.Series):
        return pd.Series(out, index=adv_dollars.index)
    if isinstance(adv_dollars, pd.DataFrame):
        return pd.DataFrame(out, index=adv_dollars.index, columns=adv_dollars.columns)
    return out

def round_trip_cost_bps(adv_dollars):
    return config.COMMISSION_BPS + half_spread_bps(adv_dollars)

def cost_fraction(adv_dollars):
    """Round-trip cost as a fraction of notional (not bps). NaN ADV -> illiquid default."""
    return round_trip_cost_bps(adv_dollars) / 10_000.0
```

## backtest.py

```
"""Wire the pieces together: events -> R_hat/G_t -> overlay on the frozen
TSMOM base book, plus the oracle-cap upper bound.
"""
import numpy as np
import pandas as pd

from . import config
from . import events as events_mod
from . import scheduling
from . import signal as signal_mod
from . import tsmom

def weekly_return(daily_returns: pd.Series) -> pd.Series:
    """Compound daily returns within each ISO (Mon-Fri, W-FRI) week."""
    week_period = daily_returns.index.to_period(f"W-{config.REBALANCE_WEEKDAY}")
    return (1.0 + daily_returns.fillna(0.0)).groupby(week_period).prod() - 1.0

def weekly_sharpe(weekly_returns: pd.Series) -> float:
    r = weekly_returns.dropna()
    if len(r) < 2 or r.std(ddof=1) == 0:
        return np.nan
    return r.mean() / r.std(ddof=1) * np.sqrt(52)

def apply_overlay(panel, base_weights: pd.DataFrame, g_t_daily: pd.Series, apply_costs: bool = True):
    """Scale the whole base book by G_t (Friday-updated, applied to the
    following ISO week -- same cadence as the base book's own rebalance)."""
    g_applied = scheduling.friday_lag_apply(g_t_daily.to_frame("g")).iloc[:, 0]
    tilted_weights = base_weights.mul(g_applied, axis=0)
    returns = tsmom.portfolio_returns(panel, tilted_weights, apply_costs=apply_costs)
    return tilted_weights, returns, g_applied

def run_cell(panel, q: int, tau: int, kappa: float, base_weights: pd.DataFrame,
             base_returns: pd.Series, apply_costs: bool = True):
    """One (q, tau, kappa) grid cell: events -> R_hat/G_t -> tilted book."""
    returns = panel.simple_returns()
    events, x_t = events_mod.build(returns, q=q)
    r_hat, g_t, lam_t = signal_mod.build(x_t, q=q, tau=tau, kappa=kappa)
    tilted_weights, tilted_returns, g_applied = apply_overlay(panel, base_weights, g_t, apply_costs)
    return {
        "q": q, "tau": tau, "kappa": kappa,
        "x_t": x_t, "r_hat": r_hat, "g_t": g_t, "lam_t": lam_t,
        "g_applied": g_applied,
        "tilted_weights": tilted_weights, "tilted_returns": tilted_returns,
    }

def oracle_g(g_weekly: pd.Series, base_week_return: pd.Series) -> pd.Series:
    """Rearrangement-inequality oracle: reassigns the SAME multiset of G
    values (identical empirical distribution -- "matchad fordelning") to
    weeks in the order that maximizes sum(g * r), i.e. perfect foresight
    within the primary's own realized G distribution. Costs ignored (this
    is a theoretical ceiling, not a tradeable variant)."""
    common = pd.concat([g_weekly, base_week_return], axis=1, keys=["g", "r"]).dropna()
    if len(common) < 2:
        return pd.Series(dtype=float)
    g_sorted = np.sort(common["g"].to_numpy())
    order = common["r"].sort_values().index # ascending return order
    oracle = pd.Series(g_sorted, index=order)
    return oracle.reindex(common.index)

def oracle_cap_test(base_returns: pd.Series, g_weekly: pd.Series) -> dict:
    base_week = weekly_return(base_returns)
    g_lagged_weekly = g_weekly.shift(1) # the multiplier a week actually trades under
    oracle_multiplier = oracle_g(g_lagged_weekly, base_week)
    common = pd.concat([base_week, oracle_multiplier], axis=1, keys=["r", "g"]).dropna()
    oracle_returns = common["r"] * common["g"]
    base_sr = weekly_sharpe(common["r"])
    oracle_sr = weekly_sharpe(oracle_returns)
    increment = oracle_sr - base_sr
    return {
        "base_sr": base_sr, "oracle_sr": oracle_sr, "increment": increment,
        "passes": bool(np.isfinite(increment) and increment >= config.ORACLE_CAP_MIN_SR_INCREMENT),
    }
```

## twins.py

```
"""Null-baseline twins, quantile-mapped to the primary's own unconditional
G distribution -- "Dammluckans matchade-bredd-krav oversatt till matchad
exponering: bara timingen far skilja."

T1 -- voltarget twin: (sigma*/sigma_hat_t)^kappa. Amplitude-based exposure
control, the mechanism the brief argues Smittotalet beats structurally.
T2 -- count-EWMA twin: raw smoothed event-count level, no ratio/branching
structure ("den brutala" -- kills R_hat if it's just disguised
count-EWMA).
T3 -- broadcast twin (a la Runraden's T4): identical events->Cori pipeline,
but run on the sleeve's own aggregated [return] as a single synthetic
series instead of pooling per-asset event indicators. Tests whether
cross-asset pooling (breadth) adds anything over one aggregate signal.
"""
import numpy as np
import pandas as pd

from . import config
from . import events as events_mod
from . import signal as signal_mod

def quantile_map_to(reference: pd.Series, raw: pd.Series) -> pd.Series:
    """Map `raw`'s time-ordering onto `reference`'s empirical value
distribution: same marginal CDF as `reference`, only the timing comes
from `raw`. This is the "matched exposure, timing-only" null construction.
"""
    common = pd.concat([reference, raw], axis=1, keys=["ref", "raw"]).dropna()
    ref_sorted = np.sort(common["ref"].to_numpy())
    n = len(ref_sorted)
    if n < 2:
        return pd.Series(np.nan, index=raw.index)
    rank = common["raw"].rank(method="average").to_numpy() # 1..n, no pct scaling
    positions = rank - 1.0 # exact 0..n-1 when no ties
    mapped = np.interp(positions, np.arange(n), ref_sorted)
    return pd.Series(mapped, index=common.index).reindex(raw.index)

def voltarget_twin(base_returns: pd.Series, g_primary: pd.Series, kappa: float,
vol_lookback: int = config.TSMOM_VOL_LOOKBACK) -> pd.Series:
    """T1: (sigma*/sigma_hat_t)^kappa, sigma* = the base book's own
unconditional (full-sample) realized vol, sigma_hat_t = rolling realized
vol of the same book. Quantile-mapped onto g_primary's distribution."""
    sigma_hat = base_returns.rolling(vol_lookback).std() * np.sqrt(config.TRADING_DAYS_YEAR)
    sigma_star = base_returns.std() * np.sqrt(config.TRADING_DAYS_YEAR)
    raw = (sigma_star / sigma_hat) ** kappa
    return quantile_map_to(g_primary, raw)

def count_ewma_twin(x_t: pd.Series, g_primary: pd.Series, tau: int, kappa: float) -> pd.Series:
    """T2: raw smoothed event-count level, no branching-ratio structure."""
    ewma = x_t.fillna(0.0).ewm(span=tau, min_periods=tau).mean()
    baseline = ewma.mean()
    raw = (baseline / ewma) ** kappa
    return quantile_map_to(g_primary, raw)

def broadcast_aggregate_series(returns: pd.DataFrame) -> pd.Series:
    """Equal-weighted mean [return] across the universe -- the sleeve's own
aggregated-return proxy, a la Runraden's T4 build_market_basket."""
    return returns.abs().mean(axis=1, skipna=True)

def broadcast_twin(returns: pd.DataFrame, g_primary: pd.Series, q: int, tau: int, kappa: float):
    """T3: identical events->Lambda->R_hat->G pipeline, run on a single
aggregated [return] series instead of per-asset pooled events."""
    agg = broadcast_aggregate_series(returns)
    thresh = agg.shift(1).rolling(config.EVENT_LOOKBACK, min_periods=config.EVENT_LOOKBACK).quantile(q / 100.0)
    x_agg = (agg > thresh).astype(float)
    x_agg[thresh.isna()] = np.nan
    r_hat, g_raw, _ = signal_mod.build(x_agg, q=q, tau=tau, kappa=kappa)
    return quantile_map_to(g_primary, g_raw)
```

## battery.py

```
"""Redundancy screen: is R_hat just a monotone transform of the count-EWMA/
vol-clustering twins ("dør mot count-EWMA-tvillingen i Steg 0")?

Two checks, both required to survive, per config.REDUNDANCY_R2_KILL /
config.REDUNDANCY_MIN_DELTA_R2 (the Runraden post-mortem lesson: pair every
null-percentile-style comparison with an absolute-magnitude floor, not just
a threshold that can itself sit at zero):

(a) Direct redundancy: regress R_hat_t on [count_ewma_t, garch_var_proxy_t].
R^2 > 0.5 -> R_hat is redundant with known twins, killed.
(b) Incremental forward-looking value: regress forward realized vol_{t+h} on
controls alone vs controls+R_hat. Incremental delta-R^2 must clear
REDUNDANCY_MIN_DELTA_R2 AND be NW-significant, or it's killed.

GARCH-persistence proxy (DECLARED approximation, same house convention as
the ADV-bucket spread caveat): a RiskMetrics-style EWMA(lambda=0.94) variance
of the sleeve's own aggregated |return| series, used as a stand-in for
fitted per-asset GARCH(1,1) persistence pooled across the universe -- not a
literal MLE GARCH fit.
"""
import numpy as np
import pandas as pd
import statsmodels.api as sm

from . import config
from . import twins as twins_mod

def garch_var_proxy(returns: pd.DataFrame, lam: float = 0.94) -> pd.Series:
    agg = twins_mod.broadcast_aggregate_series(returns)
    return (agg ** 2).ewm(alpha=1 - lam, min_periods=20).mean()

def _ols_r2(y: pd.Series, X: pd.DataFrame):
    df = pd.concat([y.rename("y"), X], axis=1).dropna()
    if len(df) < 30:
        return np.nan, None
    Xc = sm.add_constant(df.drop(columns="y"))
    model = sm.OLS(df["y"], Xc).fit(cov_type="HAC", cov_kws={"maxlags": 5})
    return model.rsquared, model

def build_battery_frame(r_hat: pd.Series, x_t: pd.Series, returns: pd.DataFrame, tau: int,
                        forward_horizon: int = 21) -> pd.DataFrame:
    count_ewma = x_t.fillna(0.0).ewm(span=tau, min_periods=tau).mean()
    garch = garch_var_proxy(returns)
    agg = twins_mod.broadcast_aggregate_series(returns)
    fwd_vol = agg.rolling(forward_horizon).std().shift(-forward_horizon) * np.sqrt(config.TRADING_DAYS_YEAR)
    return pd.DataFrame({
        "r_hat": r_hat, "count_ewma": count_ewma, "garch_var_proxy": garch, "fwd_vol": fwd_vol,
    })

def redundancy_screen(battery: pd.DataFrame) -> dict:
    df = battery.dropna()
    direct_r2, _ = _ols_r2(df["r_hat"], df[["count_ewma", "garch_var_proxy"]])

    controls_r2, _ = _ols_r2(df["fwd_vol"], df[["count_ewma", "garch_var_proxy"]])
    full_r2, full_model = _ols_r2(df["fwd_vol"], df[["count_ewma", "garch_var_proxy", "r_hat"]])
    delta_r2 = full_r2 - controls_r2 if np.isfinite(full_r2) and np.isfinite(controls_r2) else np.nan
    nw_t = float(full_model.tvalues["r_hat"]) if full_model is not None else np.nan

    direct_kill = np.isfinite(direct_r2) and direct_r2 > config.REDUNDANCY_R2_KILL
    incremental_kill = (
        not np.isfinite(delta_r2)
        or delta_r2 < config.REDUNDANCY_MIN_DELTA_R2
        or not np.isfinite(nw_t)
        or abs(nw_t) < 1.96
    )
    return {
        "direct_r2": direct_r2,
        "controls_r2": controls_r2,
        "full_r2": full_r2,
        "delta_r2": delta_r2,
        "nw_t_r_hat": nw_t,
        "direct_kill": bool(direct_kill),
        "incremental_kill": bool(incremental_kill),
        "killed": bool(direct_kill or incremental_kill),
    }
```

## episodes.py

```
"""Episode counting and binding-variation checks on R_hat_t / G_t.

"Dispersionsnull + episodrakning: R_hat's tidsvariation maste overskrida
blockshuffle-nullens band ... och >= 10 distinkta superkritiska episoder IS
(R_hat>1 >= 5 dagar, >= 21 dagars separation); bindande variation: andel
veckor med G < 0.7 i [5%, 30%]."""

import numpy as np
import pandas as pd

from . import config

def superkritiska_episodes(r_hat: pd.Series, min_run_days: int = config.EPISODE_MIN_RUN_DAYS,
                           min_separation_days: int = config.EPISODE_MIN_SEPARATION_DAYS) -> list:
    """Distinct episodes where R_hat_t > 1 for >= min_run_days consecutive
    valid days, with >= min_separation_days between the END of one episode
    and the START of the next to count as distinct."""
    r = r_hat.dropna()
    above = r > 1.0
    episodes = []
    start = None
    for i, (date, is_above) in enumerate(above.items()):
        if is_above and start is None:
            start = date
        elif not is_above and start is not None:
            end = above.index[i - 1]
            episodes.append((start, end))
            start = None
    if start is not None:
        episodes.append((start, above.index[-1]))

    runs = [(s, e) for s, e in episodes if (above.loc[s:e]).sum() >= min_run_days]

    distinct = []
    for s, e in runs:
        if distinct and (s - distinct[-1][1]).days < min_separation_days:
            distinct[-1] = (distinct[-1][0], e) # merge into the prior episode
        else:
            distinct.append((s, e))
    return distinct

def binding_g_low_share(g_t_weekly: pd.Series, threshold: float = config.BINDING_G_LOW_THRESHOLD) -> float:
    """Share of (weekly) G_t observations below `threshold`."""
    g = g_t_weekly.dropna()
    if not len(g):
        return np.nan
    return float((g < threshold).mean())

def episode_gate(r_hat: pd.Series, g_t_weekly: pd.Series) -> dict:
    episodes = superkritiska_episodes(r_hat)
    share = binding_g_low_share(g_t_weekly)
    lo, hi = config.BINDING_G_LOW_SHARE_RANGE
    return {
        "n_episodes": len(episodes),
        "episodes": episodes,
        "episode_count_pass": len(episodes) >= config.EPISODE_MIN_COUNT_IS,
        "binding_g_low_share": share,
        "binding_share_pass": bool(np.isfinite(share) and lo <= share <= hi),
    }
```

## nulls.py

```
"""Block-bootstrap / block-shuffle null-generating primitives.

circular_block_shuffle ported from research/dammluckan/nulls.py
(block_bootstrap_indices). Used for: (a) the dispersion null on R_hat's
excursion band ("Efterskalvsklockans check oversatt till tidsdimensionen"),
and (b) the Step-2 bootstrap CI on the net-SR increment.
"""

import numpy as np
import pandas as pd

from . import config
from . import signal as signal_mod

def block_bootstrap_indices(n_obs: int, length: int, block: int, rng: np.random.Generator) -> np.ndarray:
    n_blocks = int(np.ceil(length / block))
    starts = rng.integers(0, n_obs, size=n_blocks)
    idx = np.concatenate([np.arange(s, s + block) % n_obs for s in starts])
    return idx[:length]

def circular_block_shuffle(arr: np.ndarray, block_len: int, rng: np.random.Generator) -> np.ndarray:
    idx = block_bootstrap_indices(len(arr), len(arr), block_len, rng)
    return np.asarray(arr)[idx]

def r_hat_dispersion_null(x_t: pd.Series, q: int, tau: int, kappa: float, n_draws: int = 500,
                          block_len: int = 21, seed: int = 0) -> np.ndarray:
    """Circularly block-shuffle  $X_t$ , re-run the (fixed, frozen) Cori pipeline
    on each shuffle, and collect the resulting  $R_{\hat{}}$  dispersion (std over
    valid days). This is the null band  $R_{\hat{}}$ 's own time-variation must beat.
    """
    rng = np.random.default_rng(seed)
    x_vals = x_t.fillna(0.0).to_numpy()
    out = np.empty(n_draws)
    for i in range(n_draws):
        shuffled = circular_block_shuffle(x_vals, block_len, rng)
        x_shuf = pd.Series(shuffled, index=x_t.index)
        r_hat, _, _ = signal_mod.build(x_shuf, q=q, tau=tau, kappa=kappa)
        out[i] = r_hat.std(skipna=True)
    return out

def block_bootstrap_sharpe_ci(returns: pd.Series, n_draws: int = 1000, block_len: int = 21,
                              seed: int = 0, alpha: float = 0.10):
    """Two-sided (1-alpha) block-bootstrap CI on the annualized Sharpe of `returns`."""
    from . import metrics
    rng = np.random.default_rng(seed)
    r = returns.dropna().to_numpy()
    n = len(r)
    draws = np.empty(n_draws)
    for i in range(n_draws):
        idx = block_bootstrap_indices(n, n, block_len, rng)
        draws[i] = metrics.sharpe(pd.Series(r[idx]))
    draws = draws[np.isfinite(draws)]
    lo = np.quantile(draws, alpha / 2)
    hi = np.quantile(draws, 1 - alpha / 2)
    return lo, hi, draws
```



## grid.py

```
"""Grid = q in {90,95} x tau in {10,21} x kappa in {1,2} (8 cells), plus a
+-50% Omori-kernel-exponent robustness check OUTSIDE the grid (not a DSR
trial -- a sensitivity check on the one frozen, non-gridded parameter).
"""

import numpy as np
import pandas as pd

from . import backtest
from . import config
from . import metrics

def run_grid(panel, base_weights: pd.DataFrame, base_returns: pd.Series, is_start, is_end,
            oos_start, oos_end, apply_costs: bool = True) -> pd.DataFrame:
    rows = []
    for cell in config.GRID:
        result = backtest.run_cell(panel, cell.q, cell.tau, cell.kappa, base_weights, base_returns, apply_costs)
        tilted = result["tilted_returns"]
        is_ret = tilted.loc[is_start:is_end]
        oos_ret = tilted.loc[oos_start:oos_end]
        rows.append({
            "q": cell.q, "tau": cell.tau, "kappa": cell.kappa,
            "is_sharpe": metrics.sharpe(is_ret),
            "oos_sharpe": metrics.sharpe(oos_ret),
            "is_ann_return": metrics.ann_return(is_ret),
            "oos_ann_return": metrics.ann_return(oos_ret),
        })
    return pd.DataFrame(rows)

def sign_stability(grid_df: pd.DataFrame, col: str = "is_sharpe") -> float:
    signs = np.sign(grid_df[col].dropna())
    if not len(signs):
        return np.nan
    return float((signs == signs.mode().iloc[0]).mean())

def dsr_from_grid(daily_returns: pd.Series, grid_df: pd.DataFrame, sharpe_col: str = "is_sharpe") -> dict:
    """Feed the grid's per-cell (per-period) Sharpes as the DSR trial pool,
    same convention as omori/grid.py::dsr_from_grid."""
    n_obs = daily_returns.dropna().shape[0]
    sr_hat = metrics.sharpe(daily_returns) / np.sqrt(config.TRADING_DAYS_YEAR) # per-period scale
    sr_trials = (grid_df[sharpe_col].dropna() / np.sqrt(config.TRADING_DAYS_YEAR)).to_numpy()
    if len(sr_trials) < 2:
        return {"dsr_prob": np.nan, "dsr_excess": np.nan, "expected_max_sr": np.nan}
    return metrics.deflated_sharpe_ratio(sr_hat, n_obs, sr_trials)

def kernel_robustness_check(panel, base_weights, base_returns, cell, is_start, is_end,
                            pct: float = config.OMORI_KERNEL_ROBUSTNESS_PCT, apply_costs: bool = True):
    """Perturb the frozen Omori kernel exponent p by +-pct, rerun the
    default cell, report IS Sharpe under each perturbation. Outside the
    main grid -- not a DSR trial."""
    from . import events as events_mod
    from . import signal as signal_mod
    returns = panel.simple_returns()
    events, x_t = events_mod.build(returns, q=cell.q)
    out = {}
    for label, p in (
        ("p_low", config.OMORI_KERNEL_P * (1 - pct)),
        ("p_base", config.OMORI_KERNEL_P),
        ("p_high", config.OMORI_KERNEL_P * (1 + pct)),
    ):
        r_hat, g_t, _ = signal_mod.build(x_t, q=cell.q, tau=cell.tau, kappa=cell.kappa, p=p)
        _, tilted_returns, _ = backtest.apply_overlay(panel, base_weights, g_t, apply_costs)
        out[label] = {"p": p, "is_sharpe": metrics.sharpe(tilted_returns.loc[is_start:is_end])}
    return out
```

## metrics.py

```
"""Performance/risk statistics.

DSR/PSR follow the Bailey & Lopez de Prado (2014) formulation, copy-
structurally identical across every sibling research branch in this repo
(damluckan/metrics.py, oglegrinden/stats.py, omori/metrics.py) --
reproduced here in the same form. Newey-West t-stats use statsmodels HAC.
"""

import numpy as np
import pandas as pd
from scipy import stats as scipy_stats
import statsmodels.api as sm

from . import config

TRADING_DAYS = config.TRADING_DAYS_YEAR
EULER_MASCHERONI = 0.5772156649015329

def ann_return(returns: pd.Series) -> float:
    r = returns.dropna()
    return r.mean() * TRADING_DAYS if len(r) else np.nan

def ann_vol(returns: pd.Series) -> float:
    r = returns.dropna()
    return r.std(ddof=1) * np.sqrt(TRADING_DAYS) if len(r) > 1 else np.nan

def sharpe(returns: pd.Series) -> float:
    r = returns.dropna()
    if len(r) < 2 or r.std(ddof=1) == 0:
        return np.nan
    return r.mean() / r.std(ddof=1) * np.sqrt(TRADING_DAYS)

def max_drawdown(returns: pd.Series) -> float:
    r = returns.dropna()
    if not len(r):
        return np.nan
    curve = (1.0 + r).cumprod()
    dd = curve / curve.cummax() - 1.0
    return dd.min()

def newey_west_tstat(returns: pd.Series, lags=None) -> float:
    """HAC-robust t-stat that the mean (per-period) return is 0."""
    r = returns.dropna()
    if len(r) < 5:
        return np.nan
    if lags is None:
        lags = int(np.floor(4 * (len(r) / 100) ** (2 / 9)))
    ones = np.ones(len(r))
    model = sm.OLS(r.to_numpy(), ones).fit(cov_type="HAC", cov_kws={"maxlags": max(lags, 1)})
    return float(model.tvalues[0])

def probabilistic_sharpe_ratio(sr_hat: float, benchmark_sr: float, n_obs: int,
                               skew: float = 0.0, kurtosis: float = 3.0) -> float:
    """PSR (Bailey & Lopez de Prado 2012). sr_hat/benchmark_sr must be on the
    SAME (per-period, unannualized) scale."""
    if n_obs < 2:
        return np.nan
    excess_kurt = kurtosis - 3.0
    denom = np.sqrt(max(1e-12, 1 - skew * sr_hat + (excess_kurt / 4.0) * sr_hat ** 2))
    z = (sr_hat - benchmark_sr) * np.sqrt(n_obs - 1) / denom
    return float(scipy_stats.norm.cdf(z))

def expected_max_sharpe(sr_trials) -> float:
    """E[Max Sharpe] under N independent noise trials (extreme-value
    approximation), on the per-period scale of sr_trials."""
    sr_trials = np.asarray(sr_trials, dtype=float)
    sr_trials = sr_trials[np.isfinite(sr_trials)]
    n = len(sr_trials)
    if n < 2:
        raise ValueError("expected_max_sharpe requires at least 2 trials")
    sigma_sr = np.std(sr_trials, ddof=1)
    return sigma_sr * ((1 - EULER_MASCHERONI) * scipy_stats.norm.ppf(1 - 1.0 / n)
                      + EULER_MASCHERONI * scipy_stats.norm.ppf(1 - 1.0 / (n * np.e)))

def deflated_sharpe_ratio(sr_hat: float, n_obs: int, sr_trials, skew: float = 0.0,
                           kurtosis: float = 3.0) -> dict:
    """DSR probability (PSR against the expected-max-under-the-null
    benchmark) plus the Sharpe-scaled excess gap. sr_hat and sr_trials must
```

```

both be per-period Sharpes."""
sr0 = expected_max_sharpe(sr_trials)
psr = probabilistic_sharpe_ratio(sr_hat, sr0, n_obs, skew, kurtosis)
return {"dsr_prob": psr, "dsr_excess": sr_hat - sr0, "expected_max_sr": sr0}

def information_coefficient(signal_vals, forward_returns) -> float:
"""Spearman rank correlation between a signal and forward returns."""
df = pd.concat([pd.Series(signal_vals), pd.Series(forward_returns)], axis=1).dropna()
if len(df) < 5:
    return np.nan
rho, _ = scipy_stats.spearmanr(df.iloc[:, 0], df.iloc[:, 1])
return rho

def sub_period_boundaries(start, end, n_periods=config.N_SUBPERIODS):
start, end = pd.Timestamp(start), pd.Timestamp(end)
edges = pd.date_range(start, end, periods=n_periods + 1)
return [(edges[i], edges[i + 1]) for i in range(n_periods)]

def sub_period_sign_consistency(returns: pd.Series, start, end, n_periods=config.N_SUBPERIODS) -> dict:
"""Sign of annualized return in each of n_periods equal calendar
sub-periods vs. the full-sample sign."""
full_sign = np.sign(ann_return(returns))
boundaries = sub_period_boundaries(start, end, n_periods)
signs, ann_rets = [], []
for lo, hi in boundaries:
    sub = returns.loc[lo:hi]
    ar = ann_return(sub)
    ann_rets.append(ar)
    signs.append(np.sign(ar) if np.isfinite(ar) else 0.0)
n_consistent = sum(1 for s in signs if s == full_sign and s != 0)
consistency = n_consistent / n_periods if n_periods else np.nan
return {"full_sign": full_sign, "period_signs": signs, "period_ann_returns": ann_rets,
        "consistency": consistency, "boundaries": boundaries}

```

## run\_research.py

```
"""Full pre-registered pipeline, run in cost order exactly as specified
(cheapest kill first):

1. Oracle-cap test: perfect-foresight G (matched distribution) vs base.
2. Base-engine gate (Oeglegrinden-regeln): TSMOM sleeve must have net
   alpha IS before any overlay work.
3. Redundancy screen: R hat vs count-EWMA + GARCH-persistence twins.
4. Dispersion null + episode count.
5. Step 2: net-SR increment vs T1/T2/T3, bootstrap CI, sign consistency,
   grid sign-stability, pooled-corrected DSR.
OOS: single locked read of 2018-> on the primary cell only, only if every
   IS gate above passed.

Writes research/smittotalet/output/is_results_summary.json (and
oos_results_summary.json if IS survives). Mirrors the staged, cached
orchestration convention used by every sibling run_research.py.
"""
import json
import os

import numpy as np
import pandas as pd

from . import backtest
from . import battery
from . import config
from . import data
from . import episodes
from . import events as events_mod
from . import fetch_data
from . import grid as grid_mod
from . import metrics
from . import nulls
from . import signal as signal_mod
from . import tsmom
from . import twins as twins_mod

def _json_default(o):
    if isinstance(o, (np.floating, np.integer)):
        return o.item()
    if isinstance(o, (pd.Timestamp,)):
        return o.isoformat()
    if isinstance(o, np.bool_):
        return bool(o)
    return str(o)

def ensure_data():
    field_path = os.path.join(config.DATA_DIR, "primary_close.csv")
    if not os.path.exists(field_path):
        fetch_data.build_panel_csvs()

def run(apply_costs: bool = True, dispersion_draws: int = 200) -> dict:
    ensure_data()
    panel = data.load_primary_panel()
    panel = data.restrict(panel, start=config.HISTORY_START)

    results: dict = {"gates": {}, "config": {"grid": [c.__dict__ for c in config.GRID]}}

    # ---- Infra: solve k IS, build full-sample base book + default cell ----
    # (Building the base book and G_t is required plumbing for the oracle-cap
    # gate below, which is evaluated "netto-SR over basen" -- but it is NOT
    # itself a gate. Gate order below follows the brief's explicit
    # "kostnadsordning" literally: orakel-tak (1) BEFORE Steg-1 basmotor (2).)
    k = tsmom.solve_k_for_target_vol(panel, config.IS_START, config.IS_END)
    base_weights, base_returns = tsmom.build_base_book(panel, k, apply_costs=apply_costs)
    base_is = base_returns.loc[config.IS_START:config.IS_END]
    base_is_sharpe = metrics.sharpe(base_is)
    results["k"] = k
    results["base_is_sharpe"] = base_is_sharpe

    cell = config.DEFAULT_CELL
    returns = panel.simple_returns()
    events, x_t = events_mod.build(returns, q=cell.q)
    r_hat, g_t, lam_t = signal_mod.build(x_t, q=cell.q, tau=cell.tau, kappa=cell.kappa)
    tilted_weights, tilted_returns, g_applied = backtest.apply_overlay(panel, base_weights, g_t, apply_costs)

    # ---- Gate 1: oracle cap (cheapest, first) -----
    # IS-only, like every other gate below -- OOS (2018-> ) stays locked until
    # every IS gate has passed.
    from . import scheduling
    g_weekly = scheduling.week_end_values(g_t)
    oracle = backtest.oracle_cap_test(
```

```

base_returns.loc[config.IS_START:config.IS_END], g_weekly.loc[config.IS_START:config.IS_END]
)
results["gates"]["1_oracle_cap"] = oracle
if not oracle["passes"]:
    results["verdict"] = "DOD (Steg 1: orakeltak -- reglageklassen har inte tillrackligt tak har)"
    _write("is_results_summary.json", results)
    return results

# ---- Gate 2: base-engine alpha (Oeglegrinden-regeln) -----
gate2 = bool(np.isfinite(base_is_sharpe) and base_is_sharpe > config.BASE_ENGINE_MIN_IS_SHARPE)
results["gates"]["2_base_engine_alpha"] = {"is_sharpe": base_is_sharpe, "passes": gate2}
if not gate2:
    results["verdict"] = "DOD (Steg 2: basmotorn saknar netto-alfa IS -- Oeglegrinden-regeln)"
    _write("is_results_summary.json", results)
    return results

# ---- Gate 3: redundancy screen (IS-only) -----
batt = battery.build_battery_frame(
    r_hat.loc[config.IS_START:config.IS_END], x_t.loc[config.IS_START:config.IS_END],
    returns.loc[config.IS_START:config.IS_END], cell.tau,
)
redundancy = battery.redundancy_screen(batt)
results["gates"]["3_redundancy"] = redundancy
if redundancy["killed"]:
    results["verdict"] = "DOD (Steg 3: redundant med count-EWMA/GARCH-persistens)"
    _write("is_results_summary.json", results)
    return results

# ---- Gate 4: dispersion null + episode count -----
r_hat_is = r_hat.loc[config.IS_START:config.IS_END]
x_t_is = x_t.loc[config.IS_START:config.IS_END]
real_dispersion = float(r_hat_is.std(skipna=True))
null_dispersion = nulls.r_hat_dispersion_null(x_t_is, cell.q, cell.tau, cell.kappa,
                                              n_draws=dispersion_draws)
null_p95 = float(np.quantile(null_dispersion, 0.95))
dispersion_pass = bool(np.isfinite(real_dispersion) and real_dispersion > null_p95)

g_weekly_is = g_weekly.loc[config.IS_START:config.IS_END]
ep = episodes.episode_gate(r_hat_is, g_weekly_is)
gate4 = bool(dispersion_pass and ep["episode_count_pass"] and ep["binding_share_pass"])
results["gates"]["4_dispersion_episodes"] = {
    "real_dispersion": real_dispersion, "null_p95_dispersion": null_p95,
    "dispersion_pass": dispersion_pass, **ep, "passes": gate4,
}
if not gate4:
    results["verdict"] = "DOD (Steg 4: dispersionsnull eller episodfattigdom)"
    _write("is_results_summary.json", results)
    return results

# ---- Step 5: twins, grid, DSR -----
t1 = twins_mod.voltarget_twin(base_returns, g_t, cell.kappa)
t2 = twins_mod.count_ewma_twin(x_t, g_t, cell.tau, cell.kappa)
t3 = twins_mod.broadcast_twin(returns, g_t, cell.q, cell.tau, cell.kappa)

twin_results = {}
for name, twin_g in (("T1_voltarget", t1), ("T2_count_ewma", t2), ("T3_broadcast", t3)):
    _, twin_returns, _ = backtest.apply_overlay(panel, base_weights, twin_g, apply_costs)
    primary_is = tilted_returns.loc[config.IS_START:config.IS_END]
    twin_is = twin_returns.loc[config.IS_START:config.IS_END]
    diff = (primary_is - twin_is).dropna()
    lo, hi, _ = nulls.block_bootstrap_sharpe_ci(diff)
    increment = metrics.sharpe(primary_is) - metrics.sharpe(twin_is)
    sign_cons = metrics.sub_period_sign_consistency(diff, config.IS_START, config.IS_END)
    twin_results[name] = {
        "twin_sharpe_is": metrics.sharpe(twin_is),
        "primary_sharpe_is": metrics.sharpe(primary_is),
        "increment": increment,
        "increment_meets_bar": bool(np.isfinite(increment) and increment >= config.STEP2_MIN_SR_INCREMENT),
        "diff_bootstrap_ci90": [lo, hi],
        "sign_consistency": sign_cons["consistency"],
        "sign_consistency_meets_bar": bool(
            np.isfinite(sign_cons["consistency"]) and sign_cons["consistency"] >= config.STEP2_MIN_SIGN_CONSISTENCY
        ),
    }
results["step2_twins"] = twin_results

grid_df = grid_mod.run_grid(panel, base_weights, base_returns, config.IS_START, config.IS_END,
                           config.OOS_START, config.OOS_END, apply_costs)
sign_stab = grid_mod.sign_stability(grid_df)
dsr = grid_mod.dsr_from_grid(tilted_returns.loc[config.IS_START:config.IS_END], grid_df)

grid_sharpes = (grid_df["is_sharpe"].dropna() / np.sqrt(config.TRADING_DAYS_YEAR)).to_numpy()
pooled_trials = np.tile(grid_sharpes, config.N_EFFECTIVE_SURFACE_READS) if len(grid_sharpes) else grid_sharpes
sr_hat_period = metrics.sharpe(tilted_returns.loc[config.IS_START:config.IS_END]) / np.sqrt(config.TRADING_DAYS_YEAR)
pooled_dsr = metrics.deflated_sharpe_ratio(
    sr_hat_period, tilted_returns.loc[config.IS_START:config.IS_END].dropna().shape[0], pooled_trials
) if len(pooled_trials) >= 2 else {"dsr_prob": np.nan, "dsr_excess": np.nan, "expected_max_sr": np.nan}

results["grid"] = grid_df.to_dict(orient="records")
results["grid_sign_stability"] = sign_stab
results["dsr_own_grid"] = dsr

```

```

results["dsr_pooled_surface_corrected"] = pooled_dsr

kernel_robust = grid_mod.kernel_robustness_check(panel, base_weights, base_returns, cell,
                                                  config.IS_START, config.IS_END, apply_costs=apply_costs)
results["kernel_robustness"] = kernel_robust

step2_pass = all(
    v["increment_meets_bar"] and v["sign_consistency_meets_bar"] and v["diff_bootstrap_ci90"][0] > 0
    for v in twin_results.values()
) and bool(np.isfinite(pooled_dsr["dsr_excess"]) and pooled_dsr["dsr_excess"] >= config.DSR_OOS_KILL)
results["gates"]["5_step2"] = {"passes": step2_pass}

if not step2_pass:
    results["verdict"] = "DOD (Steg 5: klarar inte inkrement/CI/teckenkonsistens/DSR mot samtliga tvillingar)"
    _write("is_results_summary.json", results)
    return results

results["verdict"] = "OVERLEVER STEG 1-5 IS -- laser OOS (2018- )"
_write("is_results_summary.json", results)

# ---- Locked OOS read (only if every IS gate passed) -----
oos_results = _run_oos(panel, base_weights, base_returns, cell, apply_costs)
_write("oos_results_summary.json", oos_results)
results["oos"] = oos_results
return results

def _run_oos(panel, base_weights, base_returns, cell, apply_costs):
    returns = panel.simple_returns()
    events, x_t = events_mod.build(returns, q=cell.q)
    r_hat, g_t, lam_t = signal_mod.build(x_t, q=cell.q, tau=cell.tau, kappa=cell.kappa)
    _, tilted_returns, _ = backtest.apply_overlay(panel, base_weights, g_t, apply_costs)

    oos_primary = tilted_returns.loc[config.OOS_START:config.OOS_END]
    oos_base = base_returns.loc[config.OOS_START:config.OOS_END]
    return {
        "oos_start": config.OOS_START, "oos_end": config.OOS_END,
        "base_sharpe_oos": metrics.sharpe(oos_base),
        "primary_sharpe_oos": metrics.sharpe(oos_primary),
        "increment_oos": metrics.sharpe(oos_primary) - metrics.sharpe(oos_base),
        "n_episodes_oos": len(episodes.superkritiska_episodes(r_hat.loc[config.OOS_START:config.OOS_END])),
    }

def _write(filename, obj):
    os.makedirs(config.OUTPUT_DIR, exist_ok=True)
    with open(os.path.join(config.OUTPUT_DIR, filename), "w") as f:
        json.dump(obj, f, indent=2, default=json_default)

if __name__ == "__main__":
    out = run()
    print(json.dumps({"verdict": out.get("verdict")}, indent=2))

```

## tests/test\_config.py

```
from .. import config

# Runraden's own independently-built "40-ETF-EODHD-panels"
# (research/runraden/config.py:IS_UNIVERSE, flattened), reproduced here only
# to cross-check the ticker-panel overlap claimed in config.py's docstring.
_RUNRADEN_IS_UNIVERSE = [
    "SPY", "QQQ", "IWM", "DIA", "MDY",
    "XLK", "XLF", "XLE", "XLV", "XLI", "XLY", "XLP", "XLB", "XLU", "XLRE", "XLC",
    "EFA", "EEM", "VGK", "EWJ", "FXI",
    "TLT", "IEF", "SHY", "LQD", "HYG", "TIP", "BND", "AGG",
    "GLD", "SLV", "USO", "DBC", "UNG",
    "VNQ", "UUP", "FXE",
    "SMH", "ITB", "KRE",
]

def test_is_universe_has_40_tickers_no_duplicates():
    assert len(config.IS_UNIVERSE) == 40
    assert len(set(config.IS_UNIVERSE)) == 40

def test_is_universe_overlap_with_runraden_reconstruction():
    shared = set(config.IS_UNIVERSE) & set(_RUNRADEN_IS_UNIVERSE)
    assert len(shared) == 29

def test_grid_has_8_cells():
    assert len(config.GRID) == 8
    assert len(set((c.q, c.tau, c.kappa) for c in config.GRID)) == 8

def test_default_cell_is_in_grid():
    assert config.DEFAULT_CELL in config.GRID
```

## tests/test\_events.py

```
import numpy as np
import pandas as pd

from .. import events

def test_event_threshold_is_pit_no_lookahead():
    idx = pd.date_range("2020-01-01", periods=400, freq="B")
    rng = np.random.default_rng(0)
    returns = pd.DataFrame({"A": rng.normal(0, 0.01, len(idx))}, index=idx)
    thresh = events.event_threshold(returns, q=95, lookback=252)
    # threshold at t must be computable from data strictly before t: perturbing
    # returns.loc[t] must not change threshold.loc[t]
    perturbed = returns.copy()
    t = idx[300]
    perturbed.loc[t, "A"] = 999.0
    thresh_perturbed = events.event_threshold(perturbed, q=95, lookback=252)
    assert np.isclose(thresh.loc[t, "A"], thresh_perturbed.loc[t, "A"], equal_nan=True)

def test_event_matrix_extreme_value_always_flagged():
    idx = pd.date_range("2020-01-01", periods=400, freq="B")
    rng = np.random.default_rng(0)
    returns = pd.DataFrame({"A": rng.normal(0, 0.01, len(idx))}, index=idx)
    returns.iloc[350] = 5.0 # huge outlier, must exceed any reasonable percentile
    ev = events.event_matrix(returns, q=95, lookback=252)
    assert ev.iloc[350]["A"] == 1.0

def test_daily_count_sums_across_assets():
    idx = pd.date_range("2020-01-01", periods=10, freq="B")
    ev = pd.DataFrame({"A": [1, 0, 1], "B": [1, 1, 0]}, index=idx[:3])
    x_t = events.daily_count(ev)
    assert list(x_t) == [2, 1, 1]

def test_build_returns_matrix_and_counts_consistently():
    idx = pd.date_range("2020-01-01", periods=400, freq="B")
    rng = np.random.default_rng(0)
    returns = pd.DataFrame(rng.normal(0, 0.01, (len(idx), 5)), index=idx,
                           columns=list("ABCDE"))
    ev, x_t = events.build(returns, q=90)
    assert (x_t.dropna() == ev.sum(axis=1, skipna=True).loc[x_t.dropna().index]).all()
```



## tests/test\_signal.py

```
import numpy as np
import pandas as pd

from .. import signal

def test_omori_kernel_sums_to_one():
    w = signal.omori_kernel(lags=10, p=0.5758457103777312, c=1.0)
    assert w.shape == (10,)
    assert np.isclose(w.sum(), 1.0)
    assert np.all(np.diff(w) < 0) # monotone decaying

def test_branching_intensity_zero_when_no_events():
    idx = pd.date_range("2020-01-01", periods=50, freq="B")
    x_t = pd.Series(0.0, index=idx)
    w = signal.omori_kernel()
    lam = signal.branching_intensity(x_t, w)
    assert (lam.dropna() == 0.0).all()

def test_r_hat_equals_one_when_x_matches_lambda_exactly():
    """If  $X_t$  is generated to exactly match its own expected  $\Lambda_t$  on
    average,  $R_{\text{hat}}$  should hover near 1 (subcritical/critical boundary)."""
    idx = pd.date_range("2020-01-01", periods=300, freq="B")
    rng = np.random.default_rng(0)
    x_t = pd.Series(rng.poisson(3, size=len(idx)).astype(float), index=idx)
    w = signal.omori_kernel()
    lam = signal.branching_intensity(x_t, w)
    r_hat = signal.reproduction_number(x_t, lam, tau=21)
    valid = r_hat.dropna()
    assert len(valid) > 0
    # i.i.d. Poisson  $X$  has no branching structure ->  $R_{\text{hat}}$  should be near 1
    assert 0.5 < valid.median() < 2.0

def test_r_hat_rises_after_a_burst():
    idx = pd.date_range("2020-01-01", periods=200, freq="B")
    x_t = pd.Series(1.0, index=idx)
    # inject a burst: 10 elevated days, so recent  $X >$  implied  $\Lambda$  from a flat past
    x_t.iloc[100:110] = 10.0
    w = signal.omori_kernel()
    lam = signal.branching_intensity(x_t, w)
    r_hat = signal.reproduction_number(x_t, lam, tau=10)
    pre_burst = r_hat.iloc[90]
    post_burst = r_hat.iloc[109]
    assert post_burst > pre_burst

def test_tilt_clips_to_bounds():
    r_hat = pd.Series([0.01, 0.5, 1.0, 2.0, 100.0])
    g = signal.tilt(r_hat, kappa=1, low=0.3, high=1.3)
    assert g.min() >= 0.3
    assert g.max() <= 1.3

def test_tilt_monotone_decreasing_in_r_hat():
    r_hat = pd.Series(np.linspace(0.2, 5.0, 50))
    g = signal.tilt(r_hat, kappa=1)
    assert (g.diff().dropna() <= 1e-9).all()

def test_build_full_pipeline_runs():
    idx = pd.date_range("2020-01-01", periods=400, freq="B")
    rng = np.random.default_rng(1)
    x_t = pd.Series(rng.poisson(2, size=len(idx)).astype(float), index=idx)
    r_hat, g_t, lam_t = signal.build(x_t, q=95, tau=21, kappa=1)
    assert r_hat.shape == x_t.shape
    assert g_t.dropna().between(0.3, 1.3).all()
```

## tests/test\_tsmom.py

```
from dataclasses import dataclass

import numpy as np
import pandas as pd
import pytest

from .. import config
from .. import tsmom

def _synthetic_panel(n_days=1000, n_assets=6, seed=0):
    idx = pd.date_range("2010-01-01", periods=n_days, freq="B")
    rng = np.random.default_rng(seed)
    rets = rng.normal(0.0003, 0.01, size=(n_days, n_assets))
    prices = 100 * np.cumprod(1 + rets, axis=0)
    cols = [f"A{i}" for i in range(n_assets)]
    close = pd.DataFrame(prices, index=idx, columns=cols)
    volume = pd.DataFrame(rng.integers(1_000_000, 5_000_000, size=(n_days, n_assets)),
                          index=idx, columns=cols).astype(float)

    @dataclass
    class P:
        open: pd.DataFrame
        high: pd.DataFrame
        low: pd.DataFrame
        close: pd.DataFrame
        adjusted_close: pd.DataFrame
        volume: pd.DataFrame

        def simple_returns(self):
            return self.adjusted_close.pct_change()

        def dollar_volume(self):
            return self.close * self.volume

        def adv(self, lookback=config.ADV_COST_LOOKBACK):
            return self.dollar_volume().shift(1).rolling(lookback).mean()

    return P(open=close, high=close, low=close, close=close, adjusted_close=close, volume=volume)

def test_weekly_rebalanced_position_has_no_lookahead():
    panel = _synthetic_panel()
    pos = tsmom.weekly_rebalanced_position(panel)
    raw = tsmom.raw_signal(panel)
    # every row of `pos` must equal SOME earlier row of `raw` (same week's
    # Friday-close signal, shifted one week) -- not today's raw signal
    mismatches = 0
    for date in pos.index[-30:]:
        if not pos.loc[date].isna().all() and not raw.loc[date].isna().all():
            if (pos.loc[date] == raw.loc[date]).all():
                mismatches += 1
    assert mismatches == 0

def test_apply_gross_cap_never_exceeds_cap():
    panel = _synthetic_panel()
    raw = tsmom.weekly_rebalanced_position(panel)
    weights = tsmom.apply_gross_cap(raw, k=1000.0, gross_cap=2.0) # deliberately huge k
    gross = weights.abs().sum(axis=1).dropna()
    assert (gross <= 2.0 + 1e-9).all()

def test_solve_k_hits_target_vol_when_cap_never_binds():
    panel = _synthetic_panel(n_days=1200, n_assets=8)
    is_start, is_end = panel.close.index[300], panel.close.index[900]
    k = tsmom.solve_k_for_target_vol(panel, is_start, is_end, target_vol=0.10, gross_cap=200.0)
    weights = tsmom.apply_gross_cap(tsmom.weekly_rebalanced_position(panel), k, gross_cap=200.0)
    rets = tsmom.portfolio_returns(panel, weights, apply_costs=False).loc[is_start:is_end]
    achieved_vol = rets.std() * np.sqrt(config.TRAADING_DAYS_YEAR)
    assert achieved_vol == pytest.approx(0.10, rel=0.15)

def test_solve_k_respects_binding_gross_cap():
    panel = _synthetic_panel(n_days=1200, n_assets=8)
    is_start, is_end = panel.close.index[300], panel.close.index[900]
    # a very tight cap forces binding on most weeks -> achieved vol should be
    # BELOW an unreasonably high target, not blow up
    k = tsmom.solve_k_for_target_vol(panel, is_start, is_end, target_vol=5.0, gross_cap=0.5)
    weights = tsmom.apply_gross_cap(tsmom.weekly_rebalanced_position(panel), k, gross_cap=0.5)
    gross = weights.abs().sum(axis=1).dropna()
    assert (gross <= 0.5 + 1e-6).all()
```

## tests/test\_costs.py

---

```
import numpy as np
import pandas as pd

from .. import config
from .. import costs

def test_half_spread_monotone_in_liquidity():
    illiquid = costs.half_spread_bps(1e6)
    liquid = costs.half_spread_bps(1e9)
    assert illiquid > liquid

def test_round_trip_includes_commission():
    adv = 1e9
    assert costs.round_trip_cost_bps(adv) == config.COMMISSION_BPS + costs.half_spread_bps(adv)

def test_cost_fraction_scales_bps_by_1e4():
    adv = pd.Series([1e6, 1e8, 1e10])
    frac = costs.cost_fraction(adv)
    bps = costs.round_trip_cost_bps(adv)
    assert np.allclose(frac.values, bps.values / 10_000.0)

def test_nan_adv_falls_through_to_illiquid_bucket():
    val = costs.half_spread_bps(np.array([np.nan]))[0]
    assert val == config.ADV_BUCKETS[-1][1]
```

## tests/test\_twins.py

```
import numpy as np
import pandas as pd

from .. import twins

def test_quantile_map_preserves_reference_distribution():
    idx = pd.date_range("2020-01-01", periods=500, freq="B")
    rng = np.random.default_rng(0)
    reference = pd.Series(rng.uniform(0.3, 1.3, len(idx)), index=idx)
    raw = pd.Series(rng.normal(0, 1, len(idx)), index=idx)
    mapped = twins.quantile_map_to(reference, raw)
    # same empirical CDF: sorted mapped values ~= sorted reference values
    ref_sorted = np.sort(reference.reindex(mapped.dropna().index).to_numpy())
    mapped_sorted = np.sort(mapped.dropna().to_numpy())
    assert np.allclose(ref_sorted, mapped_sorted, atol=1e-6)

def test_quantile_map_preserves_raw_ordering():
    idx = pd.date_range("2020-01-01", periods=50, freq="B")
    reference = pd.Series(np.linspace(0.3, 1.3, 50), index=idx)
    raw = pd.Series(np.arange(50), index=idx) # strictly increasing
    mapped = twins.quantile_map_to(reference, raw)
    assert (mapped.diff().dropna() >= 0).all()

def test_broadcast_aggregate_series_is_mean_abs_return():
    idx = pd.date_range("2020-01-01", periods=5, freq="B")
    returns = pd.DataFrame({"A": [0.01, -0.02, 0.0, 0.03, -0.01],
                           "B": [-0.01, 0.02, 0.0, -0.03, 0.01]}, index=idx)
    agg = twins.broadcast_aggregate_series(returns)
    assert np.allclose(agg.to_numpy(), returns.abs().mean(axis=1).to_numpy())
```

## tests/test\_episodes.py

```
import numpy as np
import pandas as pd

from .. import config
from .. import episodes

def test_superkritiska_episodes_requires_min_run_length():
    idx = pd.date_range("2020-01-01", periods=30, freq="B")
    r_hat = pd.Series(0.5, index=idx)
    r_hat.iloc[10:13] = 1.5 # only 3 days > 1, below min_run_days=5
    ep = episodes.superkritiska_episodes(r_hat, min_run_days=5, min_separation_days=21)
    assert len(ep) == 0

def test_superkritiska_episodes_merges_close_runs():
    idx = pd.date_range("2020-01-01", periods=60, freq="B")
    r_hat = pd.Series(0.5, index=idx)
    r_hat.iloc[0:6] = 1.5
    r_hat.iloc[10:16] = 1.5 # close to the first run -- should merge under 21d separation
    ep = episodes.superkritiska_episodes(r_hat, min_run_days=5, min_separation_days=21)
    assert len(ep) == 1

def test_superkritiska_episodes_counts_distinct_far_apart_runs():
    idx = pd.date_range("2020-01-01", periods=120, freq="B")
    r_hat = pd.Series(0.5, index=idx)
    r_hat.iloc[0:6] = 1.5
    r_hat.iloc[60:66] = 1.5 # far apart -- should count as 2 distinct episodes
    ep = episodes.superkritiska_episodes(r_hat, min_run_days=5, min_separation_days=21)
    assert len(ep) == 2

def test_binding_g_low_share_basic():
    g = pd.Series([0.3, 0.5, 0.9, 1.2, 1.3, 0.6])
    share = episodes.binding_g_low_share(g, threshold=0.7)
    assert share == 3 / 6
```

## tests/test\_nulls.py

```
import numpy as np
import pandas as pd

from .. import nulls

def test_circular_block_shuffle_preserves_value_multiset():
    arr = np.arange(50)
    rng = np.random.default_rng(0)
    shuffled = nulls.circular_block_shuffle(arr, block_len=5, rng=rng)
    assert len(shuffled) == len(arr)
    assert set(shuffled.tolist()) <= set(arr.tolist()) # only draws from arr's own values

def test_block_bootstrap_indices_reproducible_with_seed():
    rng1 = np.random.default_rng(42)
    rng2 = np.random.default_rng(42)
    idx1 = nulls.block_bootstrap_indices(100, 100, 10, rng1)
    idx2 = nulls.block_bootstrap_indices(100, 100, 10, rng2)
    assert np.array_equal(idx1, idx2)

def test_r_hat_dispersion_null_returns_n_draws_values():
    idx = pd.date_range("2020-01-01", periods=300, freq="B")
    rng = np.random.default_rng(0)
    x_t = pd.Series(rng.poisson(2, len(idx)).astype(float), index=idx)
    out = nulls.r_hat_dispersion_null(x_t, q=95, tau=21, kappa=1, n_draws=20, block_len=10, seed=0)
    assert len(out) == 20
    assert np.all(np.isfinite(out) | np.isnan(out))

def test_block_bootstrap_sharpe_ci_brackets_point_estimate():
    from .. import metrics
    idx = pd.date_range("2020-01-01", periods=500, freq="B")
    rng = np.random.default_rng(0)
    r = pd.Series(rng.normal(0.0005, 0.01, len(idx)), index=idx)
    lo, hi, draws = nulls.block_bootstrap_sharpe_ci(r, n_draws=200, block_len=10, seed=0)
    point = metrics.sharpe(r)
    assert lo <= hi
    assert lo - 5 <= point <= hi + 5 # loose sanity bound, not a tight CI check
```

## tests/test\_metrics.py

```
import numpy as np
import pandas as pd

from .. import metrics


def test_sharpe_of_constant_positive_return_is_high():
    idx = pd.date_range("2020-01-01", periods=500, freq="B")
    r = pd.Series(0.001, index=idx)
    r.iloc[::7] += 0.0001 # tiny noise so std != 0
    assert metrics.sharpe(r) > 0


def test_max_drawdown_is_negative_or_zero():
    idx = pd.date_range("2020-01-01", periods=100, freq="B")
    rng = np.random.default_rng(0)
    r = pd.Series(rng.normal(-0.001, 0.02, 100), index=idx)
    dd = metrics.max_drawdown(r)
    assert dd <= 0


def test_deflated_sharpe_ratio_penalizes_more_trials():
    sr_trials_small = np.random.default_rng(0).normal(0, 0.05, 5)
    sr_trials_large = np.tile(sr_trials_small, 20)
    dsr_small = metrics.deflated_sharpe_ratio(0.1, 500, sr_trials_small)
    dsr_large = metrics.deflated_sharpe_ratio(0.1, 500, sr_trials_large)
    assert dsr_large["expected_max_sr"] >= dsr_small["expected_max_sr"]


def test_sub_period_sign_consistency_two_halves():
    idx = pd.date_range("2010-01-01", periods=1000, freq="B")
    r = pd.Series(0.001, index=idx)
    out = metrics.sub_period_sign_consistency(r, idx[0], idx[-1], n_periods=2)
    assert out["consistency"] == 1.0
```

## output/is\_results\_summary.json

```
{
  "gates": {
    "1_oracle_cap": {
      "base_sr": 0.571046073163753,
      "oracle_sr": 2.292960359851465,
      "increment": 1.721914286687712,
      "passes": true
    },
    "2_base_engine_alpha": {
      "is_sharpe": 0.5330209642908955,
      "passes": true
    },
    "3_redundancy": {
      "direct_r2": 0.07534471041556257,
      "controls_r2": 0.4996169331517759,
      "full_r2": 0.5029294522780994,
      "delta_r2": 0.0033125191263234655,
      "nw_t_r_hat": 2.0733355645649385,
      "direct_kill": false,
      "incremental_kill": true,
      "killed": true
    }
  },
  "config": {
    "grid": [
      {
        "q": 90,
        "tau": 10,
        "kappa": 1
      },
      {
        "q": 90,
        "tau": 10,
        "kappa": 2
      },
      {
        "q": 90,
        "tau": 21,
        "kappa": 1
      },
      {
        "q": 90,
        "tau": 21,
        "kappa": 2
      },
      {
        "q": 95,
        "tau": 10,
        "kappa": 1
      },
      {
        "q": 95,
        "tau": 10,
        "kappa": 2
      },
      {
        "q": 95,
        "tau": 21,
        "kappa": 1
      },
      {
        "q": 95,
        "tau": 21,
        "kappa": 2
      }
    ]
  },
  "k": 0.0004610853489187152,
  "base_is_sharpe": 0.5330209642908955,
  "verdict": "DOD (Steg 3: redundant med count-EWMA/GARCH-persistsens)"
}
```